
Quantitative Big Imaging - Basic segmentation

Anders Kaestner

Mar 11, 2026

CONTENTS

0.1	Image segmentation and discrete structures	1
0.2	Why do we do imaging experiments?	3
0.3	What we get from the imaging modality	7
0.4	Segmentation Approaches	10
0.5	Example Mammography	16
0.6	Segmentation	26
0.7	Segmenting Cells	30
0.8	Other image types	32
0.9	A data science approach to image processing	37
0.10	Receiver Operating Characteristic (ROC)	41
0.11	Segmenting multiple phases	45
0.12	Multiple Segmentations	47
0.13	Implementation of thresholding	48
0.14	Morphological image processing	49
0.15	Pitfalls with Segmentation	60
0.16	Summary	63

This is the lecture notes for the 4th lecture of the Quantitative big imaging class given during the spring semester 2022 at ETH Zurich, Switzerland.

0.1 Image segmentation and discrete structures

Part 1: Image formation and thresholding

Quantitative Big Imaging ETHZ: 227-0966-00L

0.1.1 Today's lecture

- Motivation
- Qualitative Approaches
- Image formation and interpretation problems
- Thresholding
 - Other types of images
 - Selecting a good threshold
- Implementation
- Morphological image processing
- Partial volume effects

Load some modules

```
from skimage.io          import imread
from skimage.color       import rgb2gray
import matplotlib.pyplot as plt
from skimage.morphology  import disk
from scipy.ndimage       import zoom
import numpy             as np
import pandas as pd
from skimage.morphology import ball
import tifffile as tiff
import plotsupport as ps

%matplotlib inline

# For the 3D rendering
# import plotly.offline as py
# from plotly.figure_factory import create_trisurf
from skimage.measure import marching_cubes
```

0.1.2 Applications

In this lecture we are going to focus on basic segmentation approaches that work well for simple two-phase materials. Segmenting complex samples like

- Beyond 1 channel of depth
- Multiple phase materials
- Filling holes in materials
- Segmenting Fossils
- Attempting to segment the cortex in brain imaging (see figure below)

can be a very challenging task. Such tasks will be covered in later lectures.

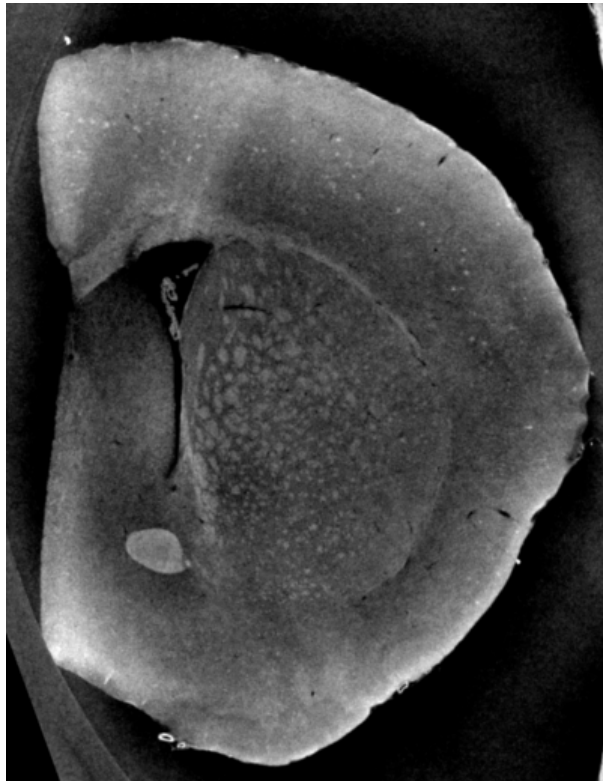


Fig. 1: An x-ray CT slice of the cortex.

- Simple two-phase materials (bone, cells, etc)
- Beyond 1 channel of depth
 - Multiple phase materials
 - Filling holes in materials
 - Segmenting Fossils
 - Attempting to segment the cortex in brain imaging

0.1.3 Literature / Useful References

- John C. Russ, [The Image Processing Handbook](#)

Models / ROC Curves

- The ROC curve [wikipedia](#)
- Julia Evans [Recalling with Precision](#)
- [Stripe's Next Top Model](#)

0.2 Why do we do imaging experiments?

There are different reasons for performing an image experiment. This often depends on in which state you are in your project.

0.2.1 Exploratory

In the initial phase, you want to learn what your sample looks like with the chosen modality. Maybe, you don't even know what is in there to see. The explorative type of experiment mostly only allows qualitative conclusions. These conclusions will however help you to formulate better hypotheses for more detailed experiments.

- To visually, qualitatively examine samples and differences between them
- No prior knowledge or expectations

0.2.2 To test a hypothesis

When you perform an experiment to test a hypothesis, you already know relatively much about your sample and want make an investigation where you can quantify characteristic features.

Quantitative assessment coupled with statistical analysis

- Does temperature affect bubble size?
- Is this gene important for cell shape and thus mechanosensation in bone?
- Does higher canal volume make bones weaker?
- Does the granule shape affect battery life expectancy?

0.2.3 Exploratory - What we are looking at?

In the exploratory analysis we inspect the images to get a first impression of the analysis situation. This inspection may be sufficient to set a diagnosis. In other cases it gives us a first glimpse to tell the complexity of the upcoming analytical tasks.

The following figures show different views of a cell. What we would expect as an illustration and what a micrograph image of a cell sample would look like. It is not always so easy to like the idealized structures with the observed ones in the measured image.

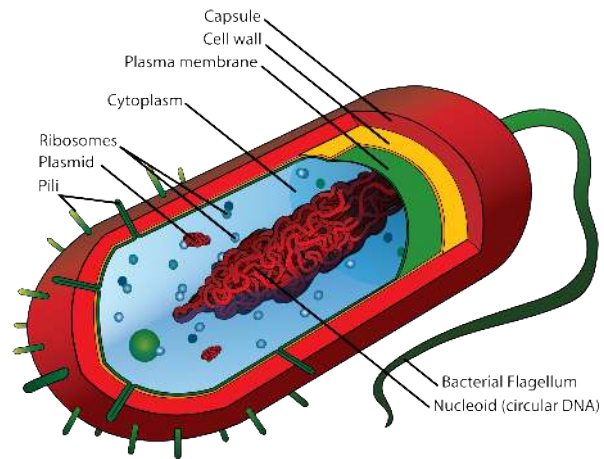


Fig. 1: Schematic illustration of a cell.

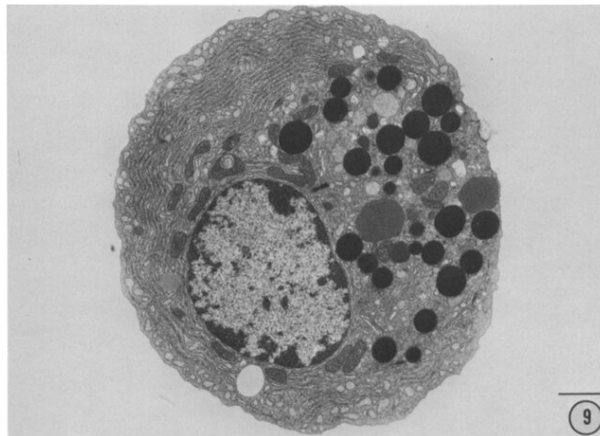


Fig. 2: Electron micrograph of an isolated chief cell. The cell contains numerous large zymogen.

0.2.4 To test a hypothesis

We perform an experiment bone to see how big the cells are inside the tissue:

We have performed an experiment that produced heaps of data to analyze. For example a using tomography.

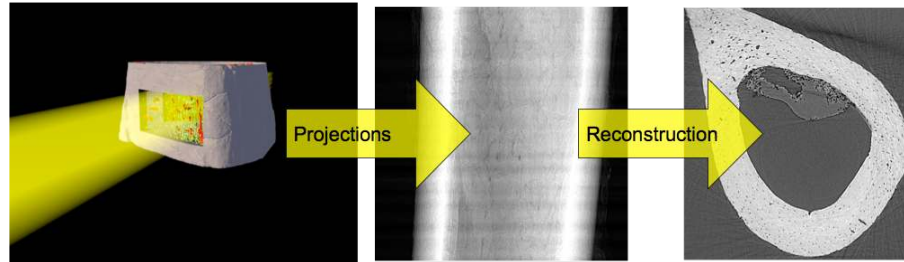
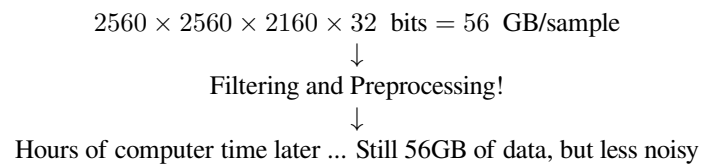


Fig. 3: Acquisition workflow to obtain CT slices of a specimen.

At the beginning we have $2560 \times 2560 \times 2160 \times 32 \text{ bits} = 56\text{GB} / \text{sample!}$ Then we apply some filtering and preprocessing to prepare the data for analysis. After 20h of computer time we still have 56GB of data (it is however nicer to work with).



Way too much data, we need to reduce

0.2.5 What did we want in the first place?

In a quantitative image analysis task we need to define metrics that relate to stated hypothesis. Some information can be obtained straight from the images. Examples are:

Single numbers:

- Volume *fraction*,
- Cell *count*,
- Average cell *stretch*,
- Cell volume *variability*

These are all **measurable** metrics!

In other cases we need to extract image metrics that are proxies for the true information.

0.2.6 Why do we perform segmentation?

In model-based analysis every step we perform, simple or complicated is related to an underlying model of the system we are dealing with

- Identify relevant regions in the images
- Many methods are available to solve the segmentation task.
- Choose wisely... *Occam's Razor* is very important here :

The simplest solution is usually the right one

Advanced methods like a Bayesian, neural networks optimized using genetic algorithms with Fuzzy logic has a much larger parameter space to explore, establish sensitivity in, and must perform much better and be tested much more thoroughly than thresholding to be justified.

The next two lectures will cover powerful segmentation techniques, in particular with unknown data.

0.2.7 Review: Filtering and Image Enhancement

This was a noise process which was added to otherwise clean imaging data

In last week's lecture we saw that images rarely are as clean as we would like and that it can, to some degree, be fixed by using filters that suppress noise and artefacts.

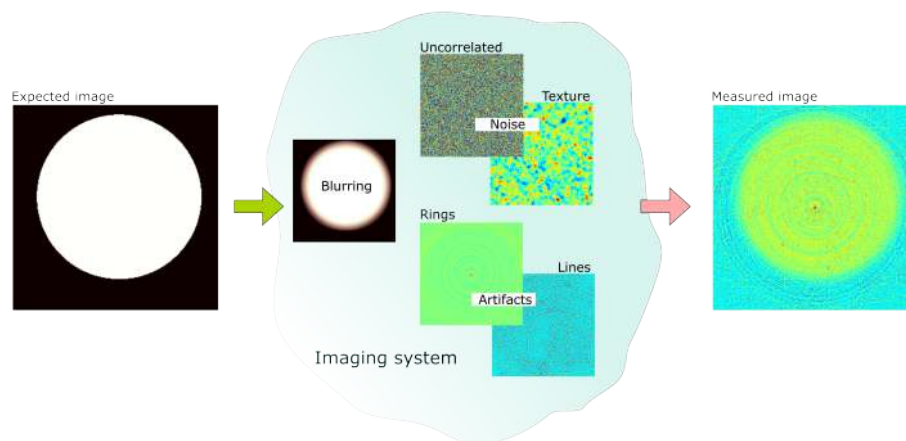


Fig. 4: Measurements rarely produce perfect images.

$$I_{measured}(x, y) = I_{sample}(x, y) + \text{Noise}(x, y)$$

- What would the perfect filter be

$$\text{Filter} * I_{sample}(x, y) = I_{sample}(x, y)$$

$$\text{Filter} * \text{Noise}(x, y) = 0$$

$$\text{Filter} * I_{measured}(x, y) = \text{Filter} * I_{real}(x, y) + \text{Filter} * \text{Noise}(x, y) \rightarrow \mathbf{I}_{sample}(\mathbf{x}, \mathbf{y})$$

Review: Filter performance

What **most filters** end up doing

$$\text{Filter} * I_{\text{measured}}(x, y) = 90\% I_{\text{real}}(x, y) + 10\% \text{Noise}(x, y)$$

Filters are not the solution to all problem, but they can get you much closer to the solution. There are gains and losses in applying a filter. The important result is that the relevant information is amplified without too much blurring and that the noise and artefacts are suppressed compared to the signal. I.e., the SNR should improve by the application of a filter.

What **bad filters** do

$$\text{Filter} * I_{\text{measured}}(x, y) = 10\% I_{\text{real}}(x, y) + 90\% \text{Noise}(x, y)$$

This case could in principle happen when you test filtering strategies and should naturally be rejected. One case which is hard to obtain good SNR is when you apply high-pass filters, because these amplify high frequencies. The noise is mostly more dominant for high frequencies.

0.3 What we get from the imaging modality

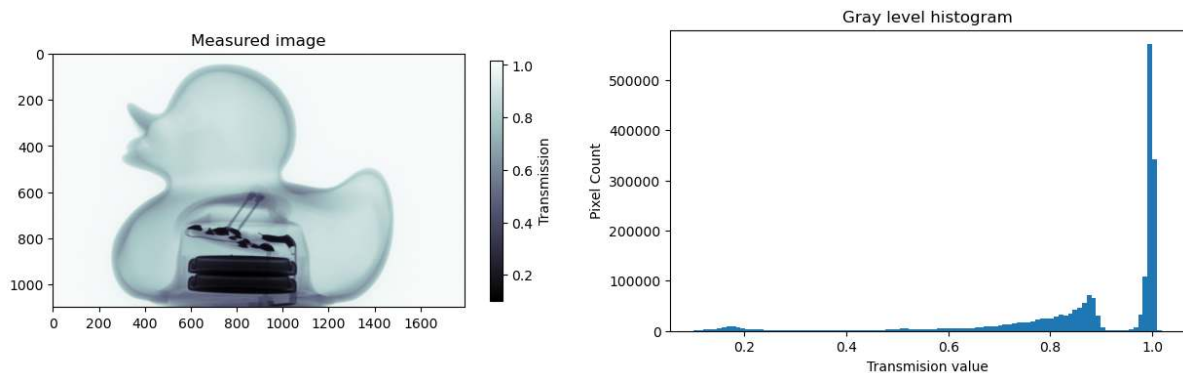
To demonstrate what we get from a modality, we load rubber duck radiograph as a toy example.

An X-ray radiograph of a rubber duck



0.3.1 Qualitative Metrics: What did people use to do?

What comes out of our detector / enhancement process



The first qualitative steps are:

- Inspect the raw image
- Modify contrast and brightness to see if new features appear
- Compute the histogram
- Describe what you see

In some cases, this may be sufficient. In this example, you could for example see that the circuit board is not perfectly fitted and would say that this is a faulty product.

Initial analysis - Identify objects by eye

The first qualitative analysis is mostly done by eye. You look at the image to describe what you see. This first assessment will help you decide how to approach the quantitative analysis task. Here, it is important to think about using words that can be translated into an image processing workflow.

- Count,
- Describe qualitatively: “batteries in the bottom”, “solder spots on PCB”, “Thin skin”

The role of initial qualitative analysis in the broader context of image analysis, particularly in disciplines that involve significant image interpretation, such as medical imaging, remote sensing, microscopy in biological sciences, and materials science.

Initial Qualitative Assessment

Observation by Eye: The first step in analyzing an image often involves a simple, yet critical, observation by the human eye. This phase is qualitative, where the analyst uses their experience, intuition, and perceptual abilities to identify patterns, anomalies, features of interest, and overall characteristics of the image. This step does not involve complex algorithms or computational tools but relies on human visual and cognitive skills.

Description and Documentation

Descriptive Analysis: After observing the image, the analyst describes what they see using precise, descriptive language. This description can include noting patterns, textures, colors, shapes, and any anomalies or features of interest. The choice of words is important; it should be detailed and objective, avoiding vague or subjective terminology as much as possible.

Bridge to Quantitative Analysis

Translating Observations into Workflow: The qualitative assessment informs the subsequent quantitative analysis. The observations made by eye must be translated into a series of steps or operations that can be performed by image processing and analysis software. This translation requires thinking critically about the descriptors used for features and patterns observed in the image. For instance, if one notices a particular texture, they must consider which computational techniques can quantify that texture—perhaps through edge detection algorithms, Fourier transforms for pattern frequency analysis, or segmentation techniques to isolate regions of interest.

Importance of Appropriate Terminology

Workflow-Friendly Vocabulary: Using terms that can be directly related to image processing operations or concepts is crucial. For example, describing a region as having a “high contrast” suggests the use of thresholding techniques for segmentation, while noting “fine, repetitive patterns” may lead to employing Fourier analysis or specific filtering techniques. The aim is to use a vocabulary that bridges the gap between qualitative observation and quantitative analysis tools.

Decision-Making for Quantitative Analysis

Guiding the Analysis Approach: The initial qualitative assessment helps in deciding the most appropriate quantitative analysis techniques. It assists in choosing the right tools, algorithms, and parameters for the analysis. For example, the presence of noise identified during the qualitative phase would influence the decision to apply noise-reduction techniques before any further quantitative analysis.

Conclusion

The process described emphasizes the iterative and complementary relationship between qualitative and quantitative analysis in image processing. Starting with a qualitative assessment by eye allows for a more informed, directed, and efficient quantitative analysis. It ensures that the selection of computational techniques is relevant to the specific features and challenges identified in the initial visual inspection, thereby improving the accuracy and relevance of the analysis outcomes. This approach is especially valuable in research and applications where precise and meaningful interpretation of images is critical.

Morphometrics

- Trace the outline of the object (or sub-structures)

0.4 Segmentation Approaches

In the introduction lecture we talked about how people approach an image analysis problem depending on their background. This is something that becomes very clear when an image is about to be segmented.

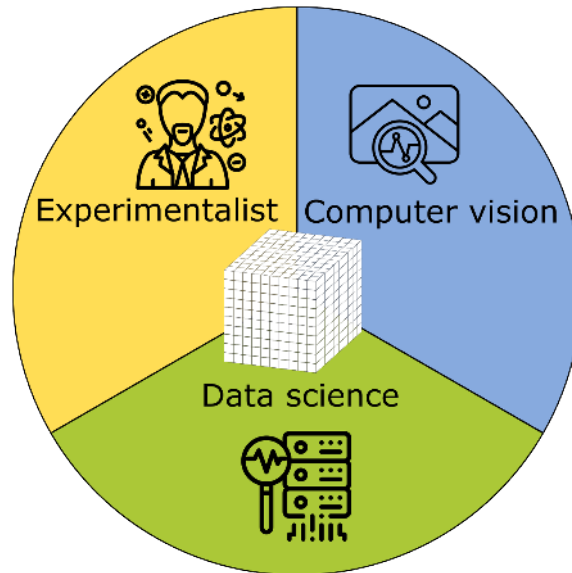


Fig. 1: Different approaches to the segmentation task depends on your background.

They match up well to the world view / perspective

0.4.1 How to approach the segmentation task

Model based segmentation

The experimentalists approached the segmentation task based on their experience and knowledge about the samples. This results in a top-down approach and quite commonly based on models fitting the real world, *what we actually can see in the images*. The analysis aims at solving the problems needed to provide answers to the defined hypothesis.

Algorithmic segmentation approach

The opposite approach is to find and use generalized algorithms that provides the results. This approach is driven by the results as the computer vision and deep learning experts often don't have the knowledge to interpret the data.

0.4.2 Model-based Analysis

The image formation process is the process to use some kind of excitation or impulse probe a sample. This requires the interaction of the four parts in the figure below.

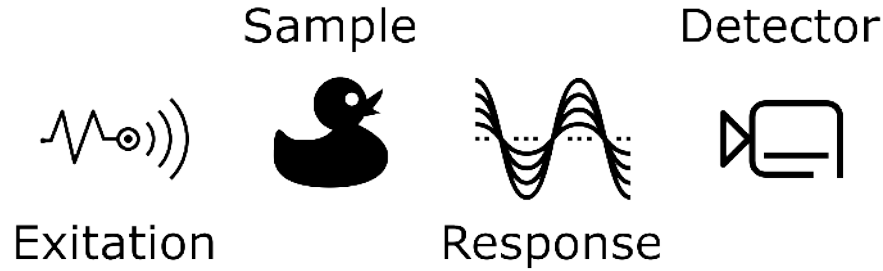


Fig. 2: The elements of the image formation process.

- **Impulses** Light, X-Rays, Electrons, A sharp point, Magnetic field, Sound wave
- **Characteristics** Electron Shell Levels, Electron Density, Phonons energy levels, Electronic, Spins, Molecular mobility
- **Response** Absorption, Reflection, Phase Shift, Scattering, Emission
- **Detection** Your eye, Light sensitive film, CCD / CMOS, Scintillator, Transducer
- Many different imaging modalities:
micro-CT to MRI to Confocal to Light-field to AFM.
- Similarities in underlying equations, but different *coefficients, units, and mechanism*

$$I_{measured}(\vec{x}) = F_{system}(I_{stimulus}(\vec{x}), S_{sample}(\vec{x}))$$

Direct Imaging (simple)

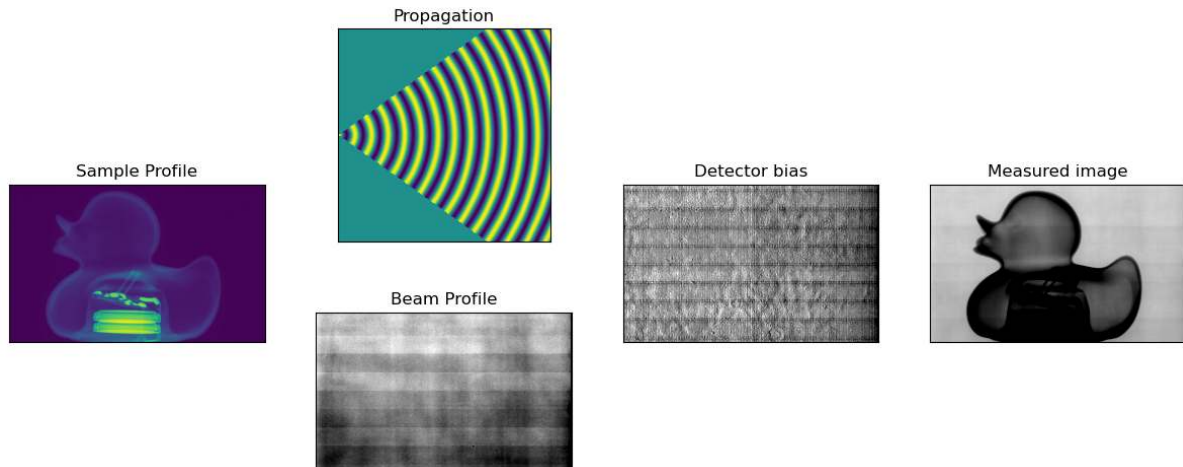
In many setups there is un-even illumination caused by incorrectly adjusted equipment and fluctuations in power and setups

$$F_{system}(a, b) = a * b$$

$$I_{stimulus} = \text{Beam}_{profile}$$

$$S_{system} = \alpha(\vec{x}) \rightarrow \alpha(\vec{x}) = \frac{I_{measured}(\vec{x})}{\text{Beam}_{profile}(\vec{x})}$$

Let's look at a radiograph where beam profile that is penetrates the sample:

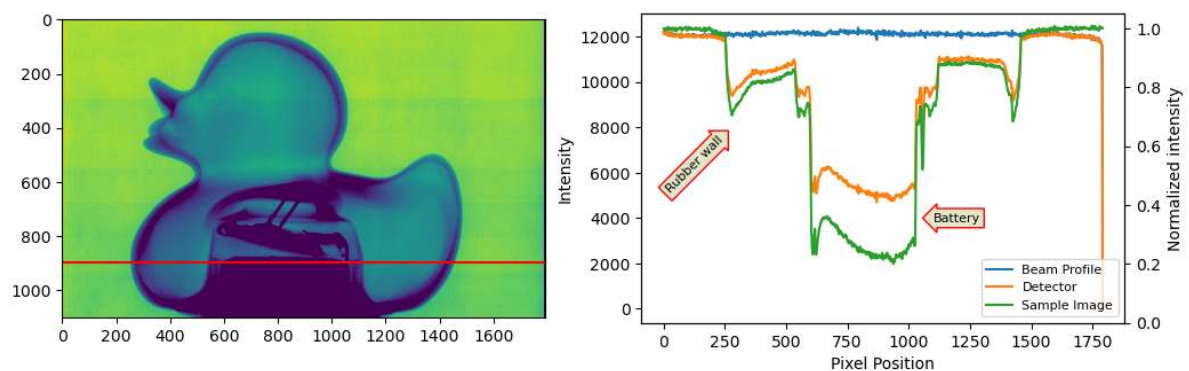


The image formation in this example involves three components that combine into the final measured image.

- The beam propagation and geometric distortion from the divergent beam. This gives a perspective projection of the sample.
- The beam profile which here has two components. The intensity distribution of the source and the amplification factors of the individual detector elements.
- The detector bias. This bias is introduced in the detector to better handle the fluctuations caused by the thermal noise in the detector.

Profiles across the image

A first qualitative analysis on images of this type is to extract line profiles to see how the transmitted intensity changes across the sample. What we can see in this particular example is that the acquired profile tapers off with the beam intensity. With this in mind, it may come clear to you that you need to normalize the images by the beam profile.



Inhomogeneous illumination

Frequently there is a gradient of the beam away from the center (as is the case of a Gaussian beam which frequently shows up for laser systems).

This can make extracting detail away from the center much harder.

Absorption Imaging (X-ray, Ultrasound, Optical)

For absorption/attenuation imaging we use **Beer-Lambert Law**

$$I_{detector} = \frac{I_{source}}{I_{stimulus}} \underbrace{e^{-\alpha d}}_{S_{sample}}$$

Different components have a different α based on

- the strength of the interaction between the light
- and the chemical / nuclear structure of the material

$$I_{sample}(x, y) = I_{source}(x, y) \cdot e^{-\int_L \alpha(l) dl}$$

$$\alpha = f(N, Z, \sigma, \dots)$$

The intensity measure at each position (x,y) in the object is the result of the line integral along L through the sample at the position of the detector position. I.e., L is located at x,y and parallel to the z-axis. The attenuation coefficient can vary along the line.

For segmentation this model is:

- there are 2 (or more) distinct components that make up the image
- these components are distinguishable by their values (or vectors, colors, tensors, ...)

A numerical transmission imaging example (1D)

In this example we create a sample with three different materials and the sample thickness $x=1.0$.

The attenuation coefficient is modelled by random models to give each measurement a realistic spread around the expected value. The attenuation coefficient is rarely an exact value in real materials. There can be fluctuations in density caused by porosity and impurities in the material.

The transmission uses Beer Lambert's law.

$$I = I_0 e^{-x \cdot \alpha}$$

	Material 1	Material 2	Material 3
α	1.0	2.0	3.0
σ_α	0.25	0.25	0.5

```
I_source = 1.0
d = 1.0
alpha_1 = np.random.normal(1, 0.25, size = 100) # Material 1
alpha_2 = np.random.normal(2, 0.25, size = 100) # Material 2
alpha_3 = np.random.normal(3, 0.50, size = 100) # Material 3

# Here, we use a dataframe to build a table of the transmissions
abs_df = pd.DataFrame([dict(alpha = c_x, material = c_mat) for c_vec, c_mat in
    zip([alpha_1, alpha_2, alpha_3],
        ['material 1', 'material 2', 'material 3']) for c_x in c_vec])

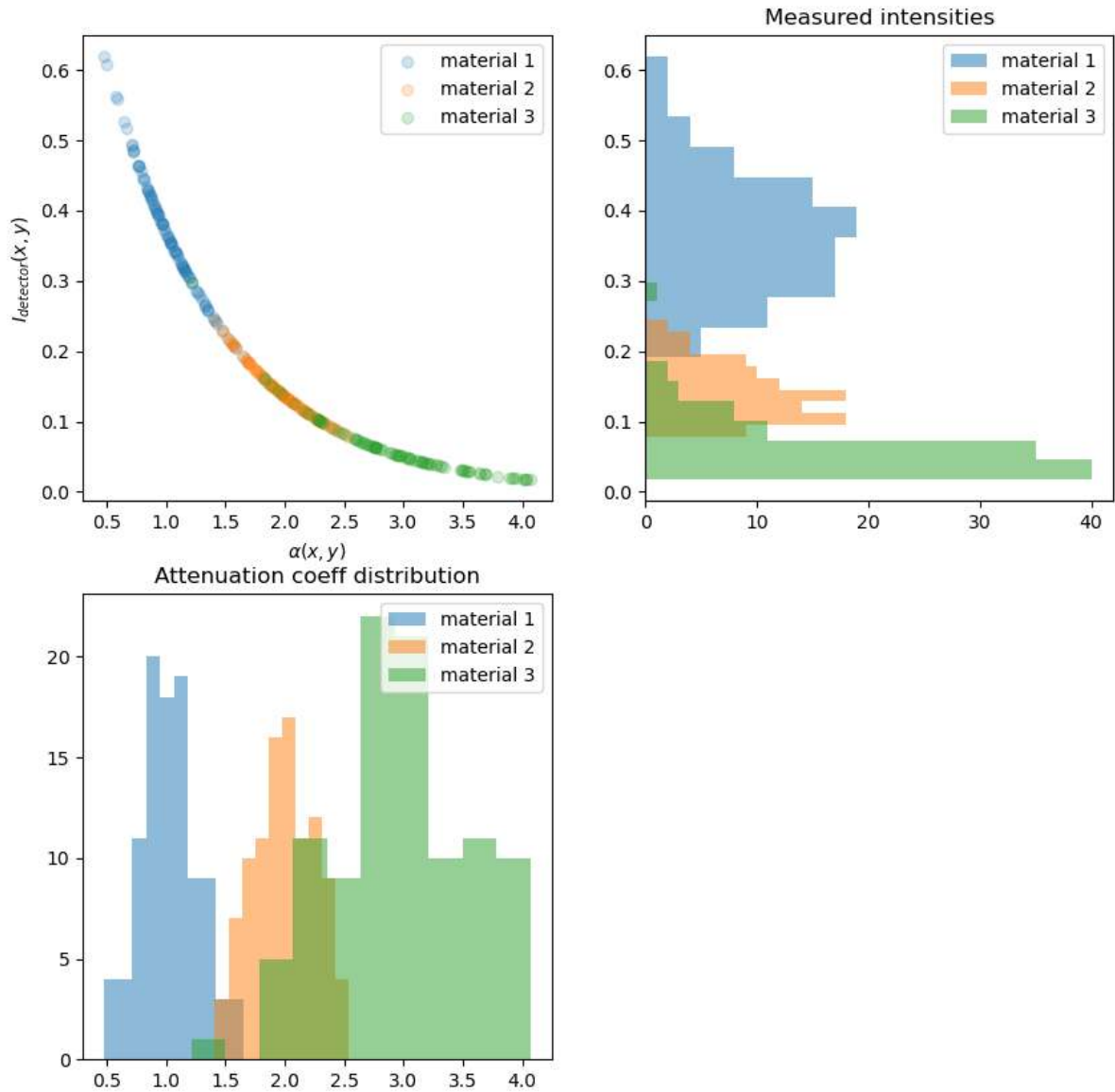
abs_df['I_detector'] = I_source*np.exp(-abs_df['alpha']*d)
abs_df.sample(5)
```

	alpha	material	I_detector
278	1.974136	material 3	0.138881
47	1.077292	material 1	0.340517
159	1.812157	material 2	0.163302
115	1.979390	material 2	0.138154
221	2.444199	material 3	0.086796

In the table, you can see that we measure different intensities on the detector depending on the material the beam is penetrating.

Plotting measured intensities

Let's now plot the intensities and attenuation coefficients and compare the outcome of our transmission experiment.



The materials are differently represented!

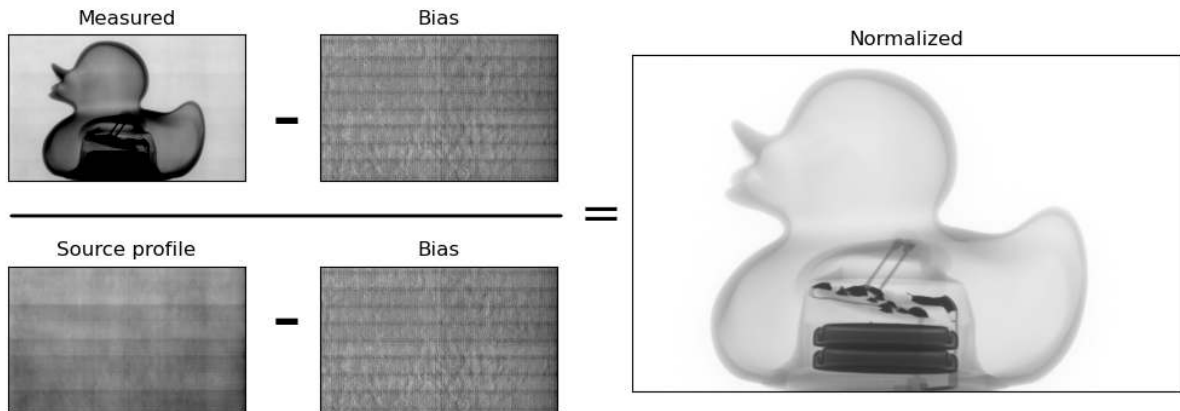
This is thanks to the exponential function in the attenuation law. large alpha values seem to be compressed in the observed attenuation image.

A further observation in this plot is that there is an overlap between the material distributions which introduces an ambiguity when we want to separate regions of different materials later.

0.4.3 Flatten a transmission image

- A transmission image can be described by Beer-Lambert's law
- Each image has a bias introduced by the detector

$$T = \frac{I_{Measured} - I_{Bias}}{I_{Illumination} - I_{Bias}} = e^{-\int \alpha(x) dx}$$



T is an image normalized between 0 and 1

$$\begin{cases} T = 1 & \text{No sample between source and detector} \\ 0 < T < 1 & \text{A sample attenuates the beam to some degree} \\ T = 0 & \text{The sample is opaque} \end{cases}$$

The α - $I_{detector}$ plot shows the curved exponential behaviour we can expect from Beer Lambert's law. Now, if we look at the histogram, we can see that distribution of attenuation coefficients doesn't really match the measured intensity. In this example, it is even so that the widths of the different materials have changed places. Great attenuation coefficient results in little transmission and small attenuation coefficient allow more of the beam to penetrate the sample.

0.5 Example Mammography

Mammographic imaging is an area where model-based absorption imaging is problematic.

Even if we assume a constant illumination (*rarely* the case),

$$I_{detector} = \frac{I_{source}}{I_{stimulus}} \underbrace{e^{-\alpha d}}_{S_{sample}}$$

$$\downarrow$$

$$I_{detector}(x, y) = I_{source} e^{-\int_0^l \alpha(x, y, z) dz}$$

The assumption that the attenuation coefficient, α , is constant is not valid. Then you see that the exponent turns into an integral along the probing ray and that α is a function of the position in the sample.

This of course leads ambiguity in the interpretation of what the pixel intensity really means.



Fig. 1: A mammography image of a breast with a malign region.

0.5.1 Problems to interpret radiography images

Specifically the problem is related to the inability to separate the

- α - attenuation
- d - thickness terms.

To demonstrate this, we model a basic breast volume as a half sphere with a constant absorption factor:

	Air	Breast tissue
$\alpha(x, y, z)$	0	0.01

→ The $\int$ then turns into a Σ in discrete space

As we only have the two conditions air and tissue. Air is however not attenuating. Therefore, the detector signal reduces to

$$I_{detector}(x, y) = I_{source} e^{-\alpha_{tissue} d_{tissue}(x, y)}$$

0.5.2 Building a breast phantom

The breast is here modelled as a half sphere of constant attenuation coefficient:

The half sphere is here created using the structure element function from scikit image morphology. More about this module later. The model is then created as a half segment of a ball with radius 50 pixels.

```
breast_mask = ball(50)[: , 50:] # This is our model
```

We can also create a volume rendering of the ball using the plotly module.

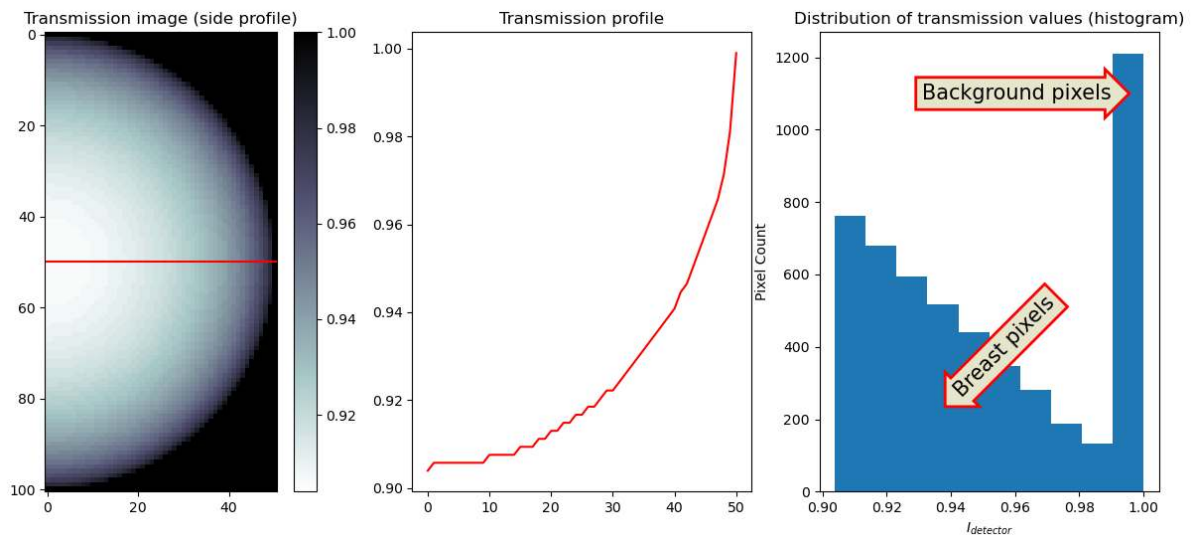
Transmission image of the breast phantom

Our first step is to simulate a transmission image of the breast. This is done by

1. Summing the attenuation coefficients times the pixel size.
2. Applying Beer-Lambert's law

This produces a 2D image of the side view of the breast.

```
breast_alpha = 1e-2 # The attenuation coefficient
pixel_size = 0.1 # The simulated detector has 1mm pixels
breast_vol = breast_alpha*breast_mask # Scale the image intensity by
    ↪ attenuation coefficient
i_detector = np.exp(-pixel_size*
    np.sum(breast_vol,axis=2)) # Compute the transmission through
    ↪ the phantom
```



The histogram shows the distribution of the transmitted intensity. Note here that all image pixels are counted in the histogram. Therefore, you get a great number of counts from the background.

Compute the thickness

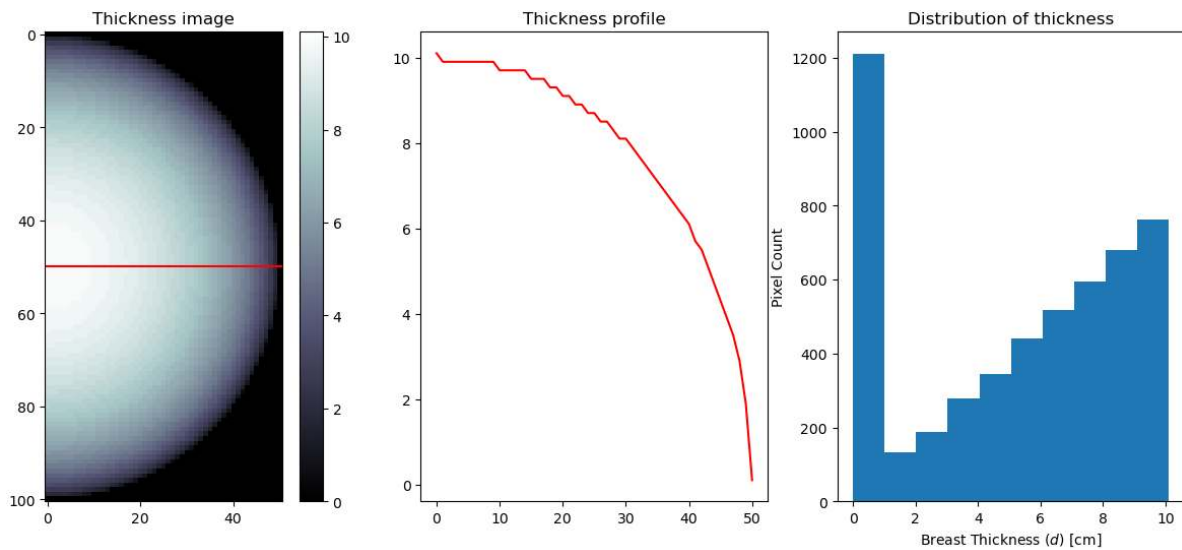
If we know that α is constant we can reconstruct the thickness d from the image:

$$d = -\log(I_{\text{detector}})/\alpha$$

This is only valid because we have air ($\alpha = 0$) as the second component in the phantom. Otherwise, if it was a denser material we would have a material mixture.

Now, let's compute the breast thickness from the transmission image:

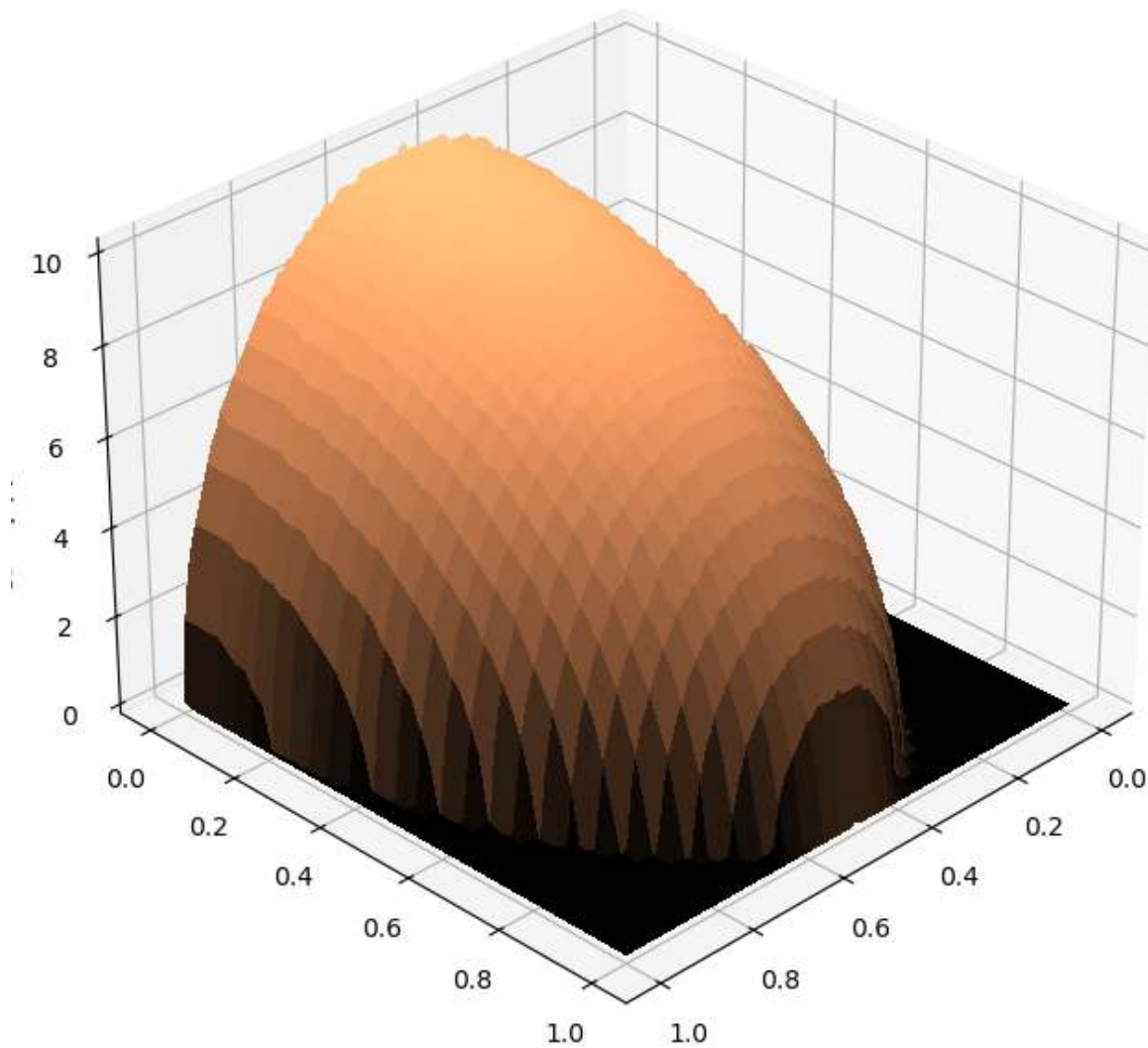
```
breast_thickness = -np.log(i_detector)/breast_alpha # Compute the thickness
```



Now, you can see that the intensity is reversed in the thickness image. An important difference is that we now have a first quantitative value that relates to the shape of the breast. The transmission image did not reveal this.

Visualizing the thickness

The thickness map appears as a parabola instead of an ellipsoid as in the geometric model. The reason is that the thickness starts at 0 cm and can only increase.



0.5.3 What if alpha is not constant?

We run into problems when the α is no longer constant.

- For example if we place a dark lump in the center of the breast.
- It is **impossible** to tell if the breast is *thicker* or if the lump inside is *denser*.

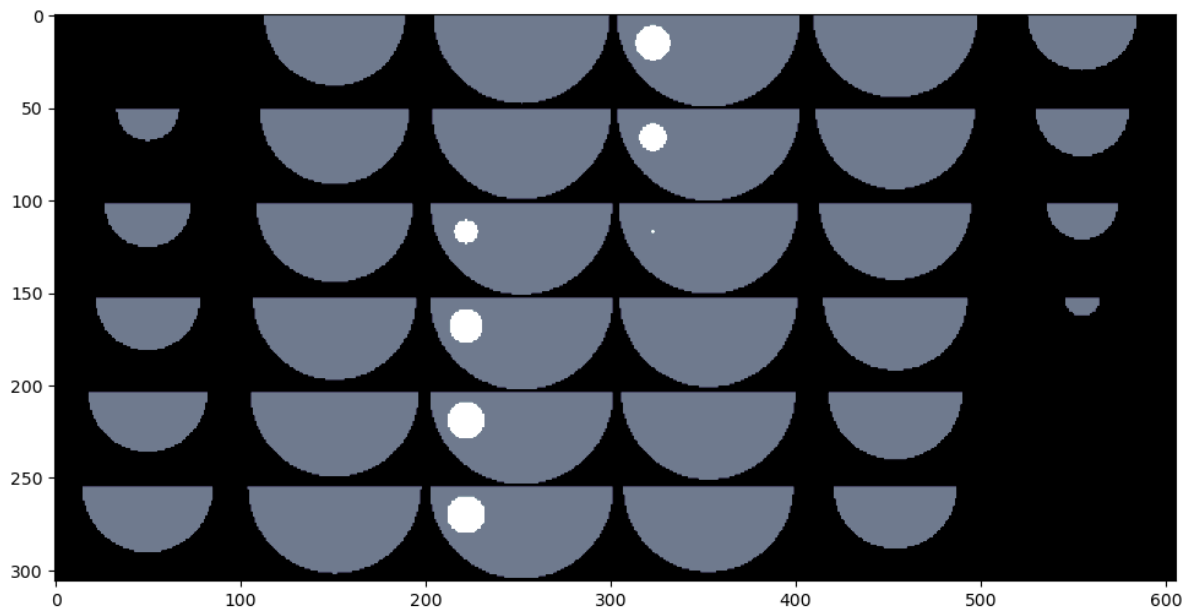
Building a new model

For the lump below we can see on the individual slices of the sample that the lesion appears quite clearly and is very strangely shaped.

	Air	Breast tissue	Lump
$\alpha(x, y, z)$	0	0.01	0.02

We model the lump as a sphere with $\alpha=0.02$ in the breast phantom. The lump appears with high contrast on the slices through the model. But we are limited to the summed profile from a transmission image.

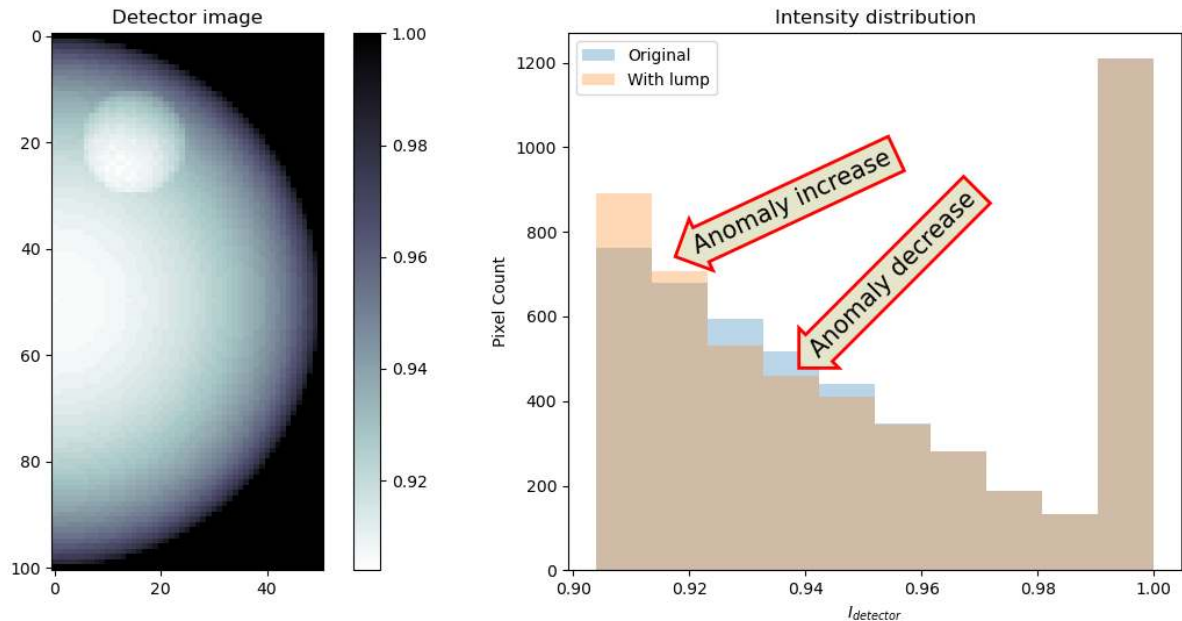
```
breast_vol2 = breast_alpha*breast_mask
alump      = 0.02
lump       = ball(10)
breast_vol2[10:31, 5:26, 40:61]=np.maximum(breast_vol2[10:31, 5:26, 40:61], lump*alump);
```



Looking at the thickness again

When we make the projection and apply Beer's Law we see that it appears as a relatively constant region in the image

```
i_detector2 = np.exp(-pixel_size*np.sum(breast_vol2,axis=2)) # Compute what the  
→detector sees
```



Comparing the histograms of the normal phantom and the phantom with a lump, we see that there is a displacement of the intensities. 1, there is an increase of pixels in the lower end of the histogram. 2, the same number of pixels are lost in the mid section of the histogram.

These are subtle changes, but give an indication that inspires further investigations.

Using the histogram is however not the usual way to make a diagnosis. It is far too imprecise because there are so many factors involved that can have an impact on the histogram shape. A further issue is that you rarely have the healthy image to compare with. Images are only made when there is a problem. Except in the case when regular monitoring is made.

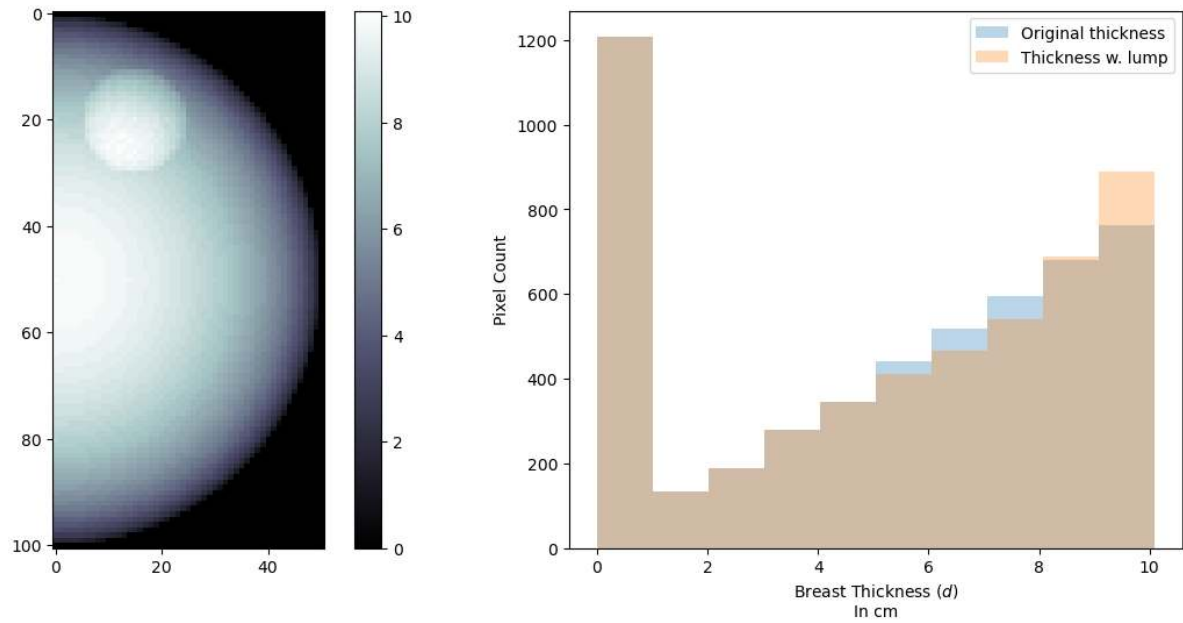
An anomaly in the thickness reconstruction

It appears as a region in the thickness reconstruction.

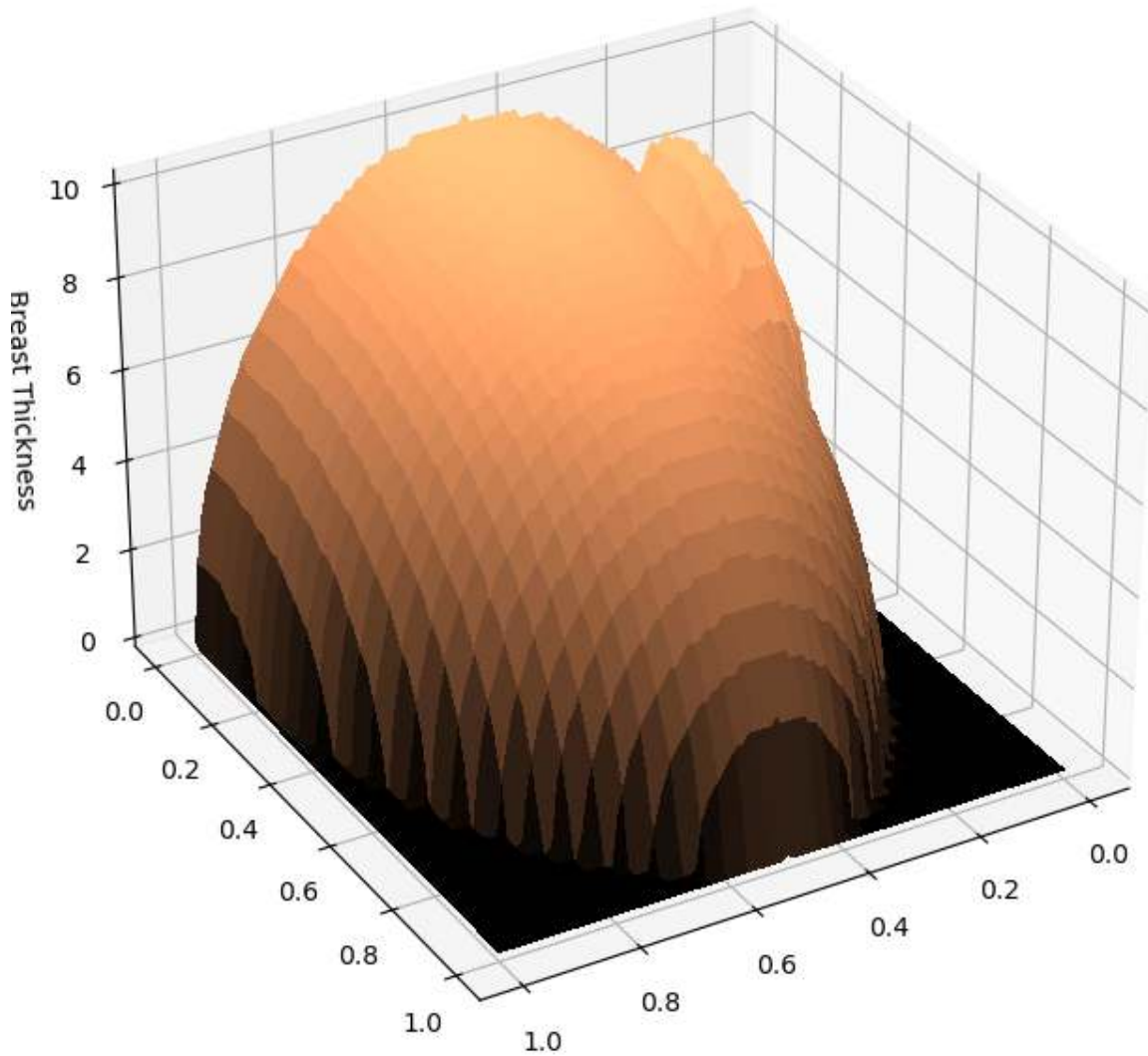
So we cannot fundamentally from this single image answer:

- is the breast oddly shaped?
- or does it have an possible tumor inside of it?

```
breast_thickness2 = -np.log(i_detector2)/1e-2
```



Looking at the thickness profile with lump



0.5.4 Ambiguity in interpreting transmission images

The thickness/density problem can be illustrated by the example in the figure. The same gray level is build up by a combination of thicknesses multiplied by their corresponding attenuation coefficient. Thus, it can happen that the same intensity is registered for regions of very different dimensions.

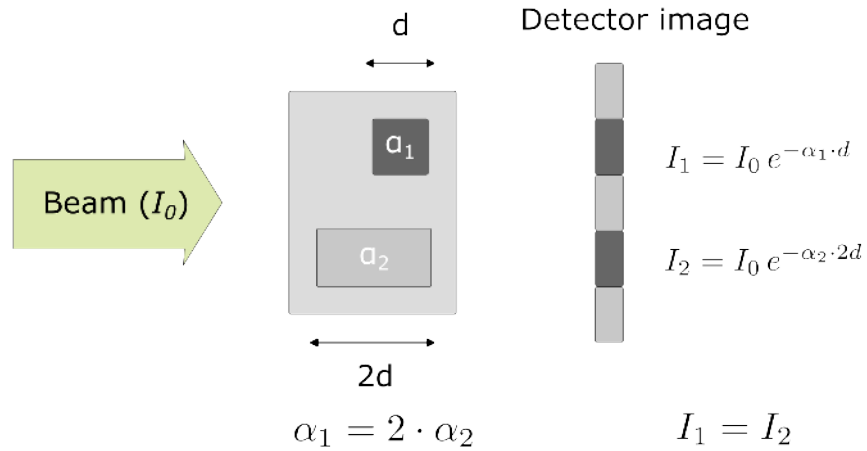


Fig. 2: The same intensity on the detector can have different origins.

Methods to reduce the ambiguity

The problem can be addressed in two ways:

1. Still use transmission imaging, but **use constant thickness**.
2. Change to an imaging method that **provides 3D information**. E.g. computed tomography.

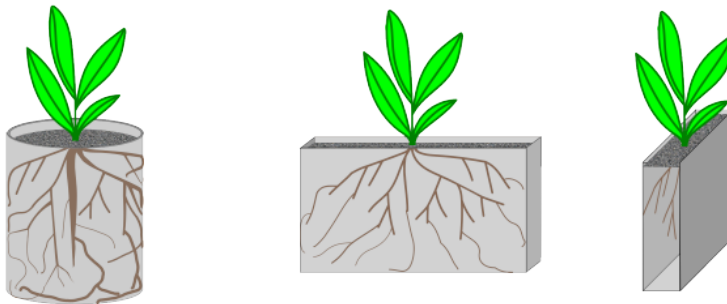


Fig. 3: Reduce the ambiguity by constraining the shape.

0.5.5 Summary of the image formation process

- Images from the detector can mostly *not* be used directly.
- Normalization is needed.
- The information in transmission images can be difficult to interpret.

0.6 Segmentation

0.6.1 Where does segmentation get us?

We can convert a decimal value or something even more complicated like

- 3 values for RGB images,
- a spectrum for hyperspectral imaging,
- or a vector / tensor in a mechanical stress field

To a single or a few discrete values:

- usually **True** or **False**,
- but for images with many phases it would be each phase, e.g. bone, air, cellular tissue.

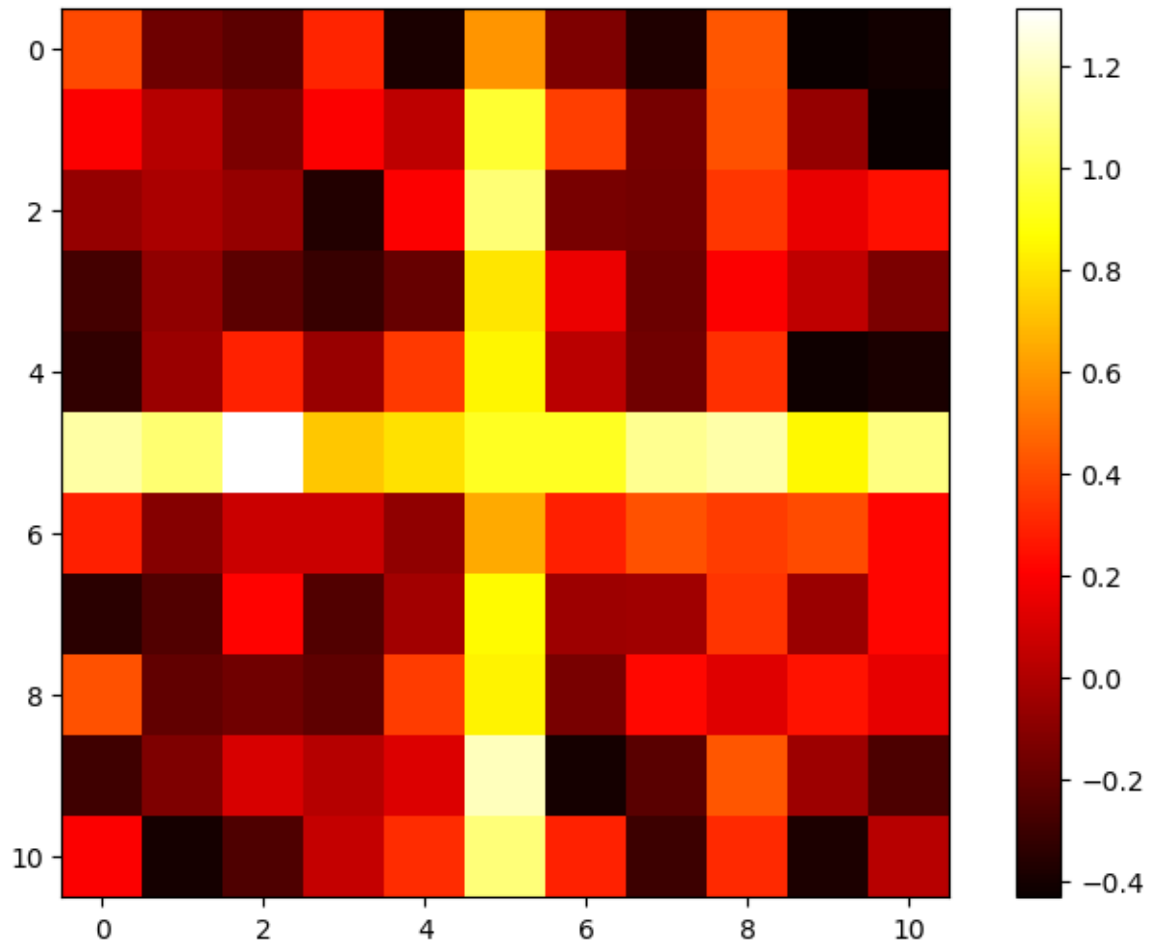
2560 x 2560 x 2160 x 32 bit = 56GB / sample → 2560 x 2560 x 2160 x **1 bit** = 1.75GB / sample

0.6.2 Basic segmentation: Applying a threshold to an image

Start out with a simple image of a cross with added noise $I(x, y) = f(x, y)$

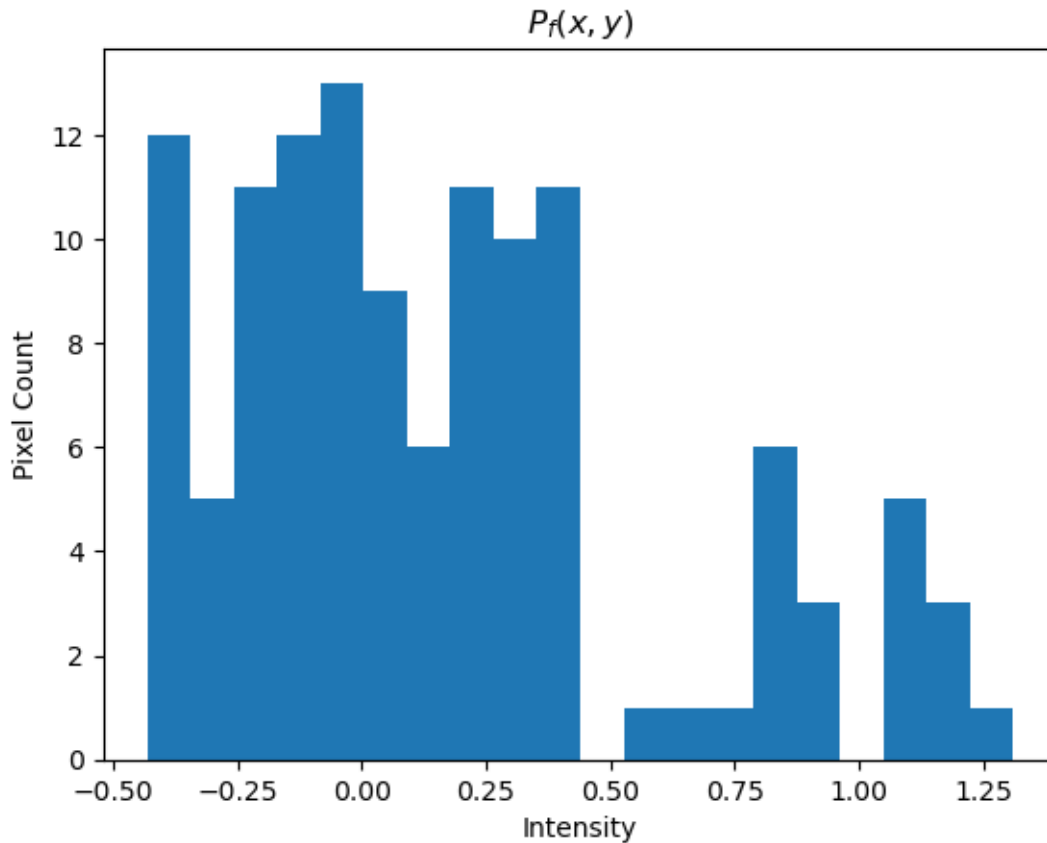
Here, we create a test image with two features embedded in uniform noise; a cross with values in the order of '1' and background with values in the order '0'. The figure below shows the image and its histogram. The histogram helps us to see how the graylevels are distributed which guides the decision where to put a threshold that segments the cross from the background.

```
nx = 5; ny = 5
xx, yy = np.meshgrid(np.arange(-nx, nx+1)/nx*2*np.pi,
                     np.arange(-ny, ny+1)/ny*2*np.pi)
cross_im = (np.abs(xx*yy) <= (2*np.pi/nx)) + 1.75*np.random.uniform(-0.25, 0.25, size = xx.shape)
```



0.6.3 The histogram

The intensity can be described with a probability density function $P_f(x, y)$



0.6.4 Applying a threshold to an image

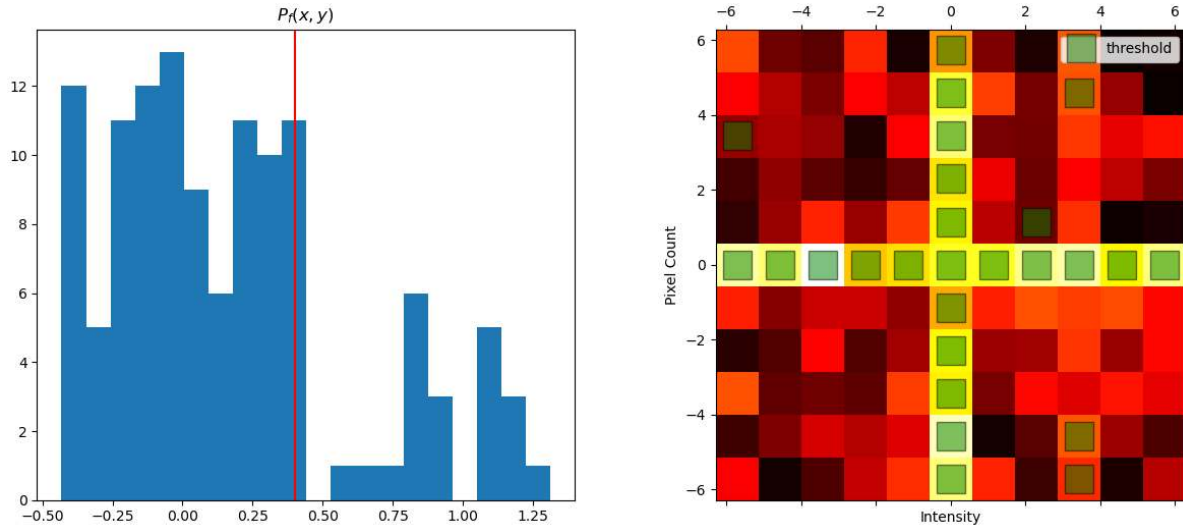
By examining the image and probability distribution function, we can *deduce* that the underlying model is a whitish phase that makes up the cross and the darkish background. This is indicated by the left side (lower intensity) that has the greater number of pixels than the right side (higher intensity).

Applying the threshold is a deceptively simple operation

$$I(x, y) = \begin{cases} 1, & f(x, y) \geq 0.40 \\ 0, & f(x, y) < 0.40 \end{cases}$$

We just compare each if each pixel is less or greater than a constant value, the threshold, and say that pixels brighter than the threshold belong to the cross.

```
threshold = 0.4
thresh_img = cross_img > threshold
```

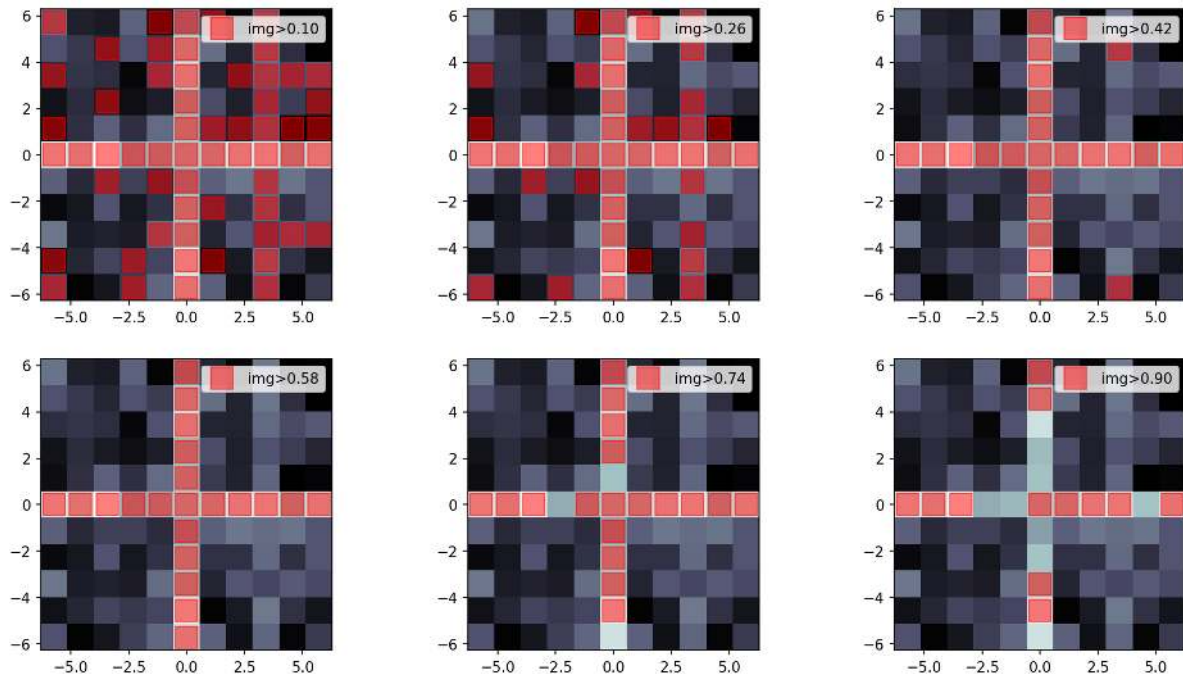


Various Thresholds

Finding the correct threshold value is crucial for the outcome of the thresholding operation. A bad choice may attribute too many or too few pixels to the intended item. A basic method to explore this is to try different thresholds and observe the effect. This can easily be done in interactive tools like ImageJ. Below, you see the effect on our cross image.

We can see the effect of choosing various thresholds

$$\gamma \in \{0.1, 0.26, 0.42, 0.58, 0.74, 0.9\}$$



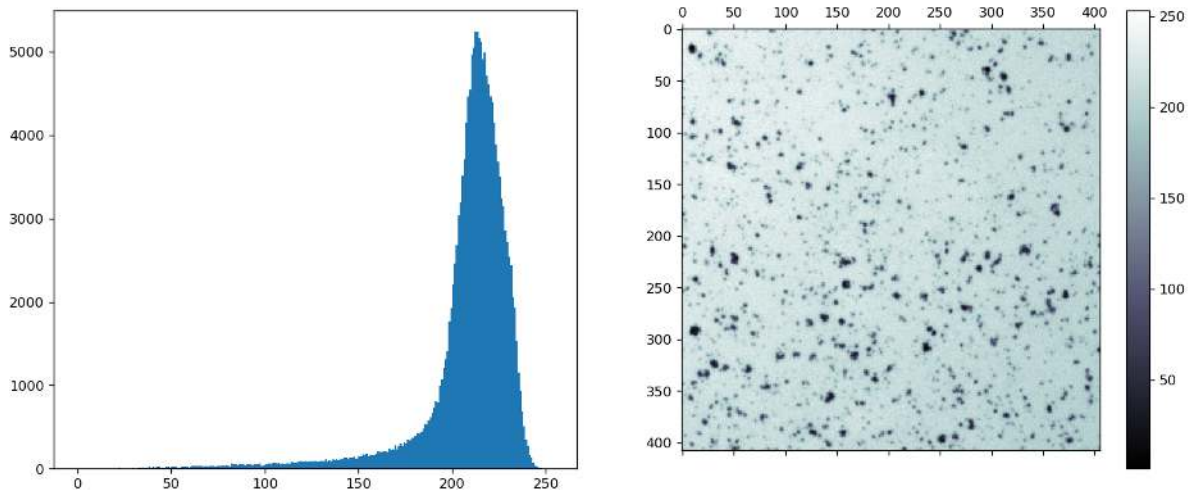
In this fabricated example we saw that thresholding can be a very simple and quick solution to the segmentation problem. Unfortunately, real data is often less obvious. The features we want to identify for our quantitative analysis are often obscured by different other features in the image. They may be part of the setup of caused by the acquisition conditions.

0.7 Segmenting Cells

We can perform the same sort of analysis with this image of cells

This time we can derive the model from the basic physics of the system

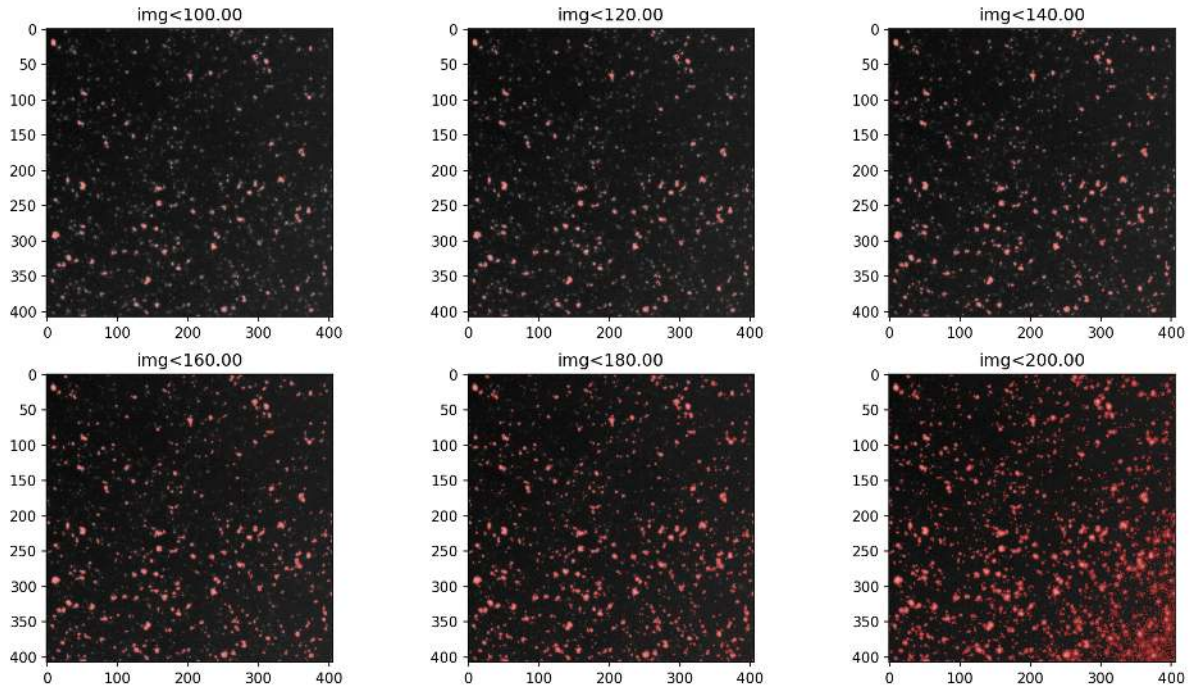
- The field is illuminated by white light of nearly uniform brightness
- Cells absorb light causing darker regions to appear in the image
- *Lighter* regions have no cells
- **Darker** regions have cells



The human eye/brain is a fantastic segmentation machine. We can easily see and label the cells in the example image. The computer, however, has a harder time for this seemingly trivial task.

0.7.1 Trying different thresholds on the cell image

Let's try the same threshold scanning approach as we did with the cross image and apply different thresholds to the cell image.



There is a graylevel gradient in the image! This can be seen in that the segmented cells in the lower right corner are larger than those in the upper left corner.

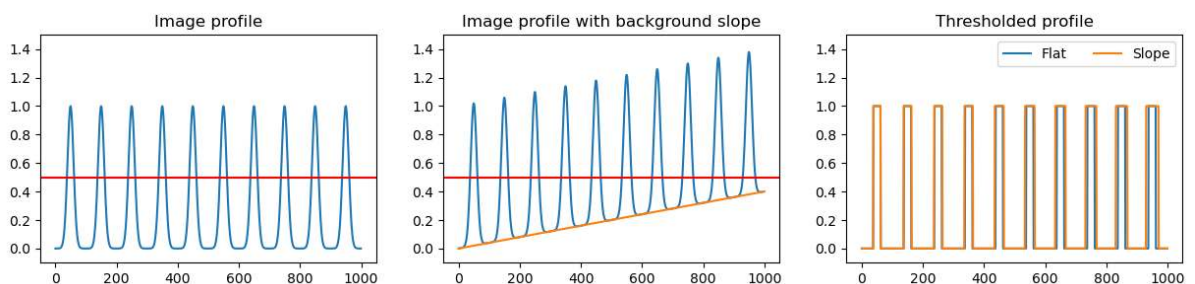
The cells regions have smooth edges which means that they are identified with different widths depending on the background intensity.

We can only speculate about the origin of this gradient, but one reason could be bad normalization to the illumination field in the microscope.

A closer look at the effect of a background gradient

We saw in the previous example that a gradient can bias the size of the detected items. This effect is mostly undesirable. Let's take a look at what happens.

In this example we reduce the problem to 1D, which would correspond to looking at an intensity profile across the image. The signal consists of several peaks with the same shape and amplitude. There is not background bias in the first case while there is a linearly increasing background in the second case. We apply the same threshold to both signals to observe what happens.



The last panel shows the resulting segmentation from the two cases. The item size is the same for all items for the unbiased signal. The increases from left to right for the biased signal. The reason is that the threshold cuts the structures lower down where the item signal is wider.

This effect means that we would introduce a bias or uncertainty in the size measurement if the slope is not corrected. We will see later in this lecture how the slope can be corrected to obtain more reliable measurements.

0.8 Other image types

While scalar images are easiest, it is possible for any type of image $I(x, y) = \vec{f}(x, y)$

In the first lecture, we already introduced images that represent more than one value per position. Examples are:

- Flow fields - a vector field representing direction and amplitude
- Spectral images - each position represents a spectral response, i.e. vectors of wavelength, intensity tuples.

Here, we create an image with vectors to show local orientation and intensities to measure the strength of a signal.

```
nx = 10
ny = 10
xx, yy = np.meshgrid(np.linspace(-2*np.pi, 2*np.pi, nx),
                     np.linspace(-2*np.pi, 2*np.pi, ny))

intensity_img = 1.5*np.abs(np.cos(xx*yy)) / (np.abs(xx*yy) + (3*np.pi/nx)) + np.random.
↳ uniform(-0.25, 0.25, size = xx.shape)

base_df = pd.DataFrame(dict(x = xx.ravel(),
                           y = yy.ravel(),
                           I_detector = intensity_img.ravel()))

base_df['x_vec'] = base_df.apply(lambda c_row: c_row['x']/np.sqrt(1e-2+np.square(c_
↳ row['x'])+np.square(c_row['y'])), 1)
base_df['y_vec'] = base_df.apply(lambda c_row: c_row['y']/np.sqrt(1e-2+np.square(c_
↳ row['x'])+np.square(c_row['y'])), 1)

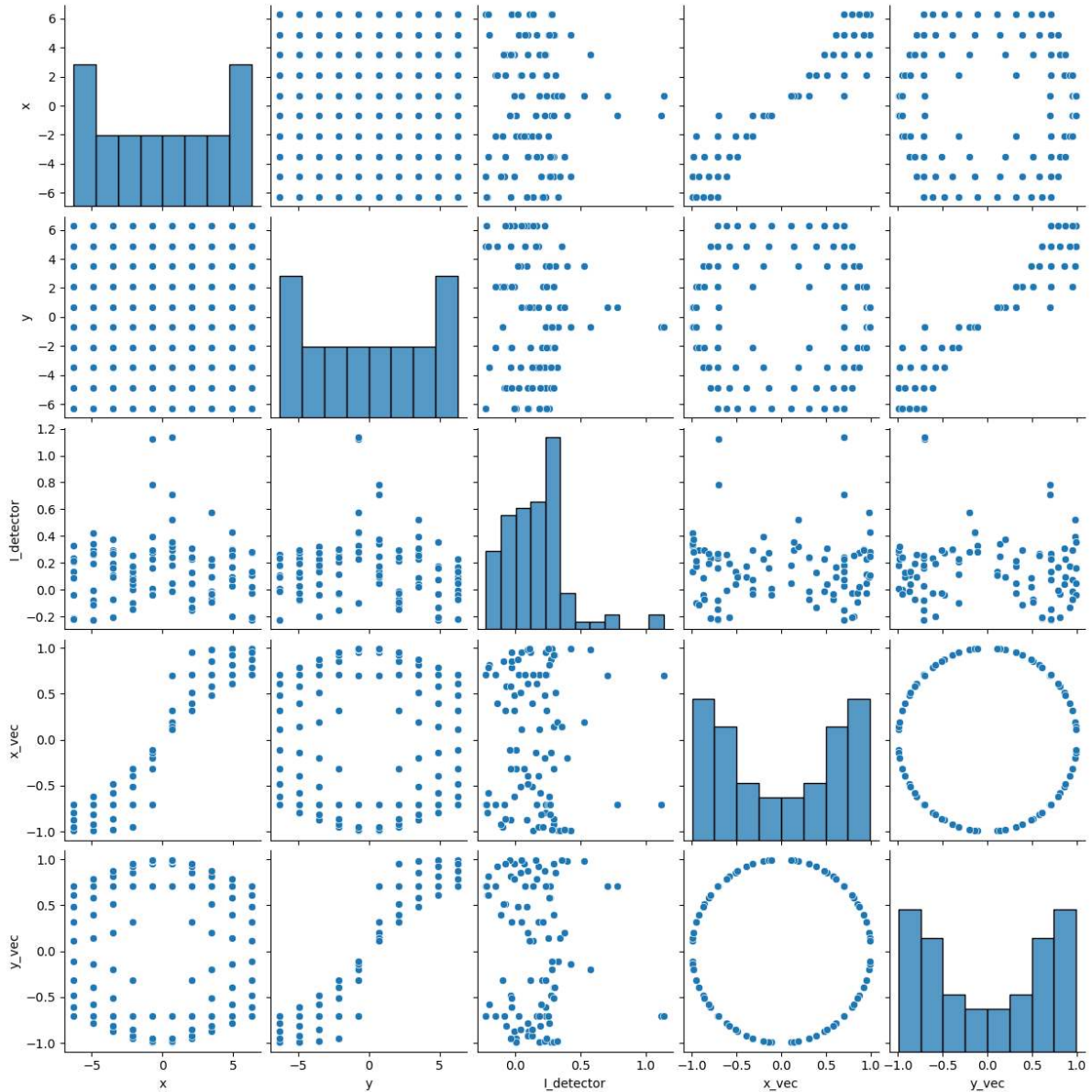
base_df.sample(5)
```

	x	y	I_detector	x_vec	y_vec
64	-0.698132	2.094395	0.294366	-0.315904	0.947712
23	-2.094395	-3.490659	0.132143	-0.514341	-0.857234
74	-0.698132	3.490659	0.398656	-0.196039	0.980194
10	-6.283185	-4.886922	0.234021	-0.789290	-0.613892
71	-4.886922	3.490659	0.267199	-0.813621	0.581158

0.8.1 Looking at colocation histograms

The colocation histogram is a powerful tool to visualize how different components are related to each other. It also called bi-variate histogram. In seaborn, there is the `pairplot` which shows colocation histograms for all combinations on the data. The diagonal is the histogram of the individual components.

```
import seaborn as sns
sns.pairplot(base_df);
```

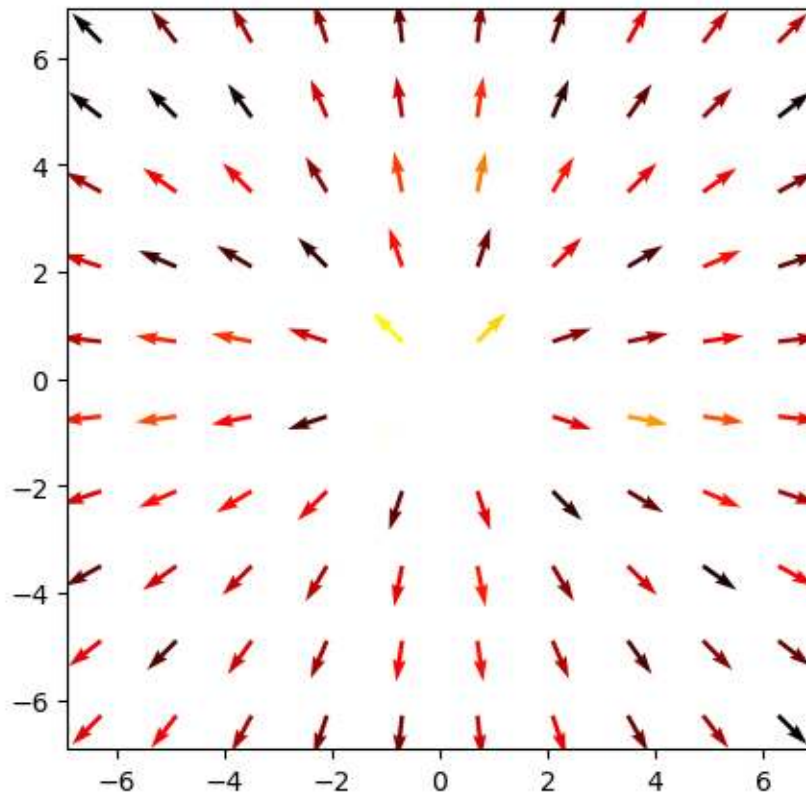


0.8.2 Vector field plot

The vector field is a common way to visualize vector data. It does however only work for small data sets like in this example, otherwise it will be too cluttered and no relevant information will be visible.

Here we map the table columns as

- x,y: Position
- x_vec, y_vec: Orientation
- I_detector: Color map



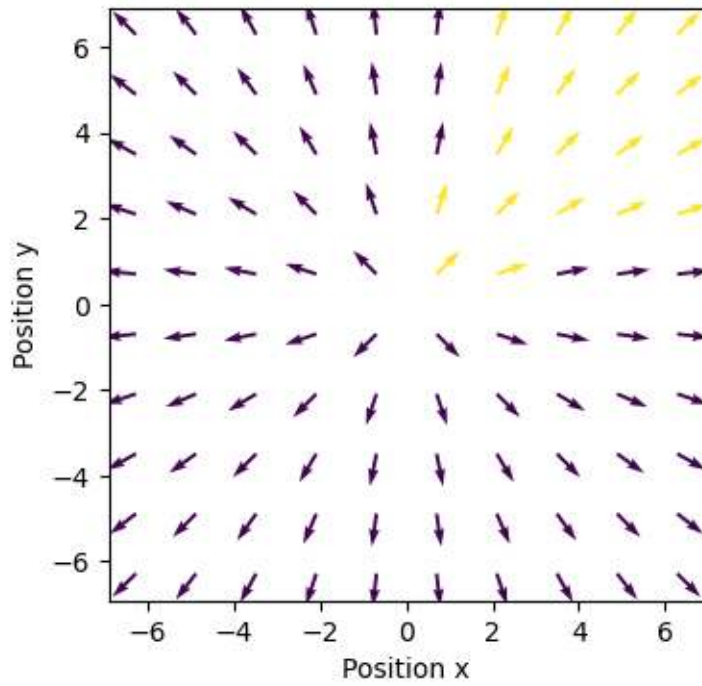
Larger vector fields can also be visualized, but then it is common to downsample the signal to allow for a reasonably uncluttered visualization.

0.8.3 Applying a threshold to vector valued image

A threshold is now more difficult to apply since there are now two distinct variables to deal with. The standard approach

$$\text{can be applied to both } I(x, y) = \begin{cases} 1, & \bar{f}_x(x, y) \geq 0.25 \text{ and} \\ & \bar{f}_y(x, y) \geq 0.25 \\ 0, & \text{otherwise} \end{cases}$$

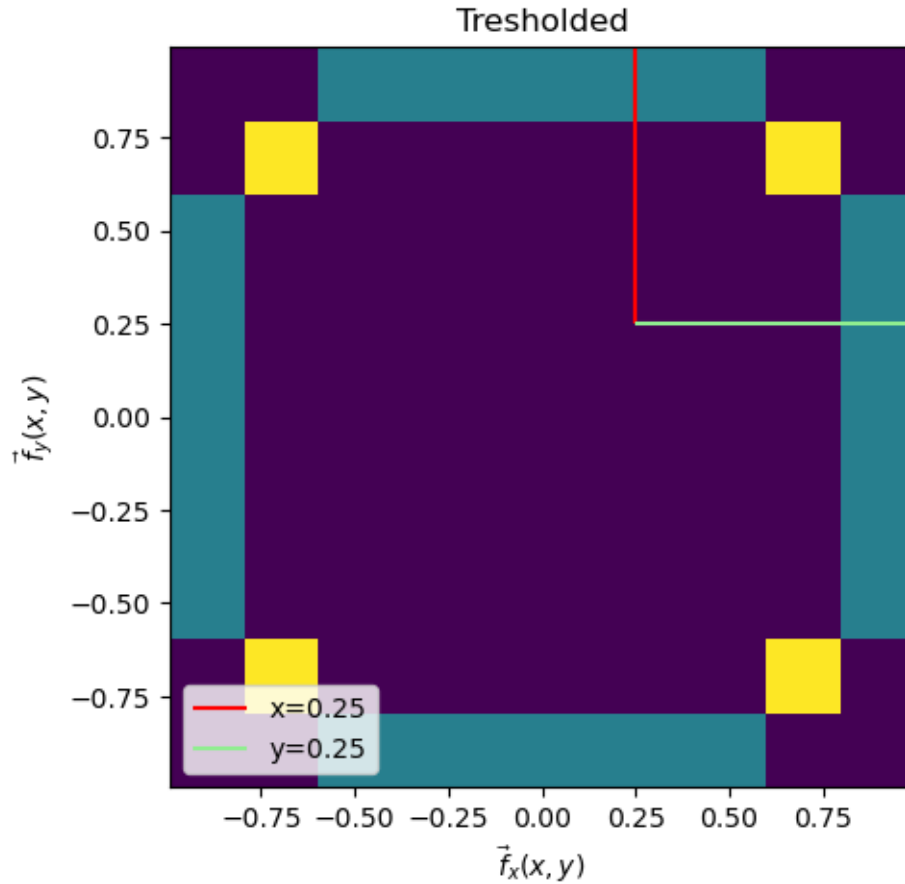
```
thresh_df = base_df.copy()
thresh_df['thresh'] = thresh_df.apply(lambda c_row: c_row['x_vec']>0.25 and c_row['y_
vec']>0.25, 1)
```



Histogram of the vectors

This can also be shown on the joint probability distribution as a bivariate histogram.

The lines here indicate the thresholded vector components.



Applying a threshold

Given the presence of two variables; however, more advanced approaches can also be investigated.

- For example we can keep only components parallel to the x axis by using the dot product.

$$I(x, y) = \begin{cases} 1, & |\vec{f}(x, y) \cdot \vec{i}| = 1 \\ 0, & \text{otherwise} \end{cases}$$

Thresholding orientations

We can tune the angular acceptance by using the fact that the scalar product can be expressed using the angle between the the vectors as

Scalar product definition

$$\vec{x} \cdot \vec{y} = |\vec{x}| |\vec{y}| \cos(\theta_{x \rightarrow y})$$

$$I(x, y) = \begin{cases} 1, & \cos^{-1}(\vec{f}(x, y) \cdot \vec{i}) \leq \theta^\circ \\ 0, & \text{otherwise} \end{cases}$$

0.8.4 Summary of basic thresholding

- Thresholding is the first approach to segmentation.
- The histogram guides the threshold selection.
- Inhomogeneous illumination and noise make the task harder.
- Non-scalar pixel values can also be thresholded.

0.9 A data science approach to image processing

0.9.1 Loading some modules

Let's load a collection of modules for this part.

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.color import rgb2gray
from skimage.io import imread
import plotsupport as ps
from sklearn.metrics import roc_auc_score
import pandas as pd
from collections import OrderedDict
from sklearn.metrics import roc_curve

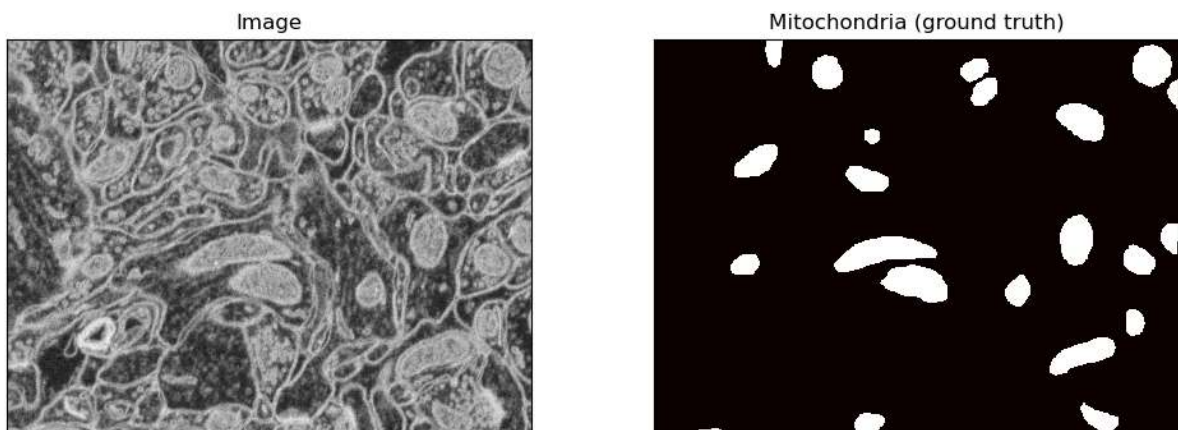
%matplotlib inline
```

0.9.2 How to approach the analysis

Segmentation and all the steps leading up to it are really a specialized type of learning problem.

Let's look at an important problem for electron microscopy imaging...

Identifying the mitochondria in the images like the one to the left in the figures below.



We want to identify which class each pixel belongs to.

What does identify mean?

- Classify the pixels in a mitochondria as *Foreground*

- Classify the pixels outside of a mitochondria as *Background*

This is a really tedious task and we want to automatize it. Here, segmentation is a good approach as we saw in the previous part of this lecture. We saw that a threshold can separate different parts of the image. The question is now how we can achieve this in a reproducible way.

0.9.3 Which values can be assigned

A bilevel segmentation has two principal outcomes: **true** and **false**.

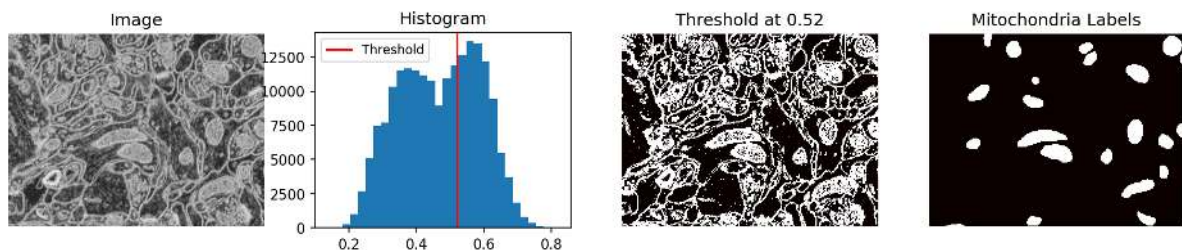
The segmentation is, however, rarely perfect. It can always happen that an outcome is put in the wrong category. Therefore, we categorize the segmented pixels into four classes:

- **True Positive**
values in the mitochondria that are classified as *Foreground*
- **True Negative**
values outside the mitochondria that are classified as *Background*
- **False Positive**
values outside the mitochondria that are classified as *Foreground*
- **False Negative**
values in the mitochondria that are classified as *Background*

Applying a threshold

We can then apply a threshold to the image to determine the number of points in each category

```
thresh = 0.52  
thresh_img = cell_img > thresh # Apply a single threshold to the image
```

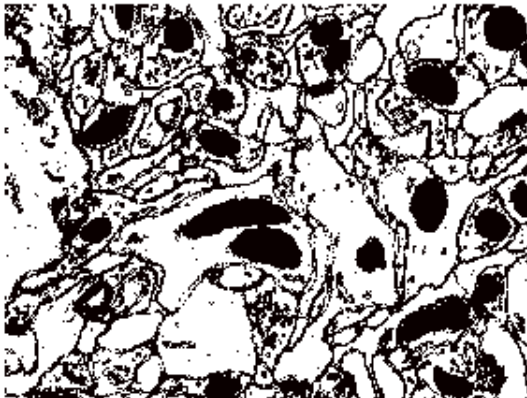


In this example we can see that it is clearly not sufficient to apply a single threshold as we have tried before. When we compare the thresholded image to the provided mask, we can see that there are plenty more structures marked as foreground and also that there are holes within the mitochondria.

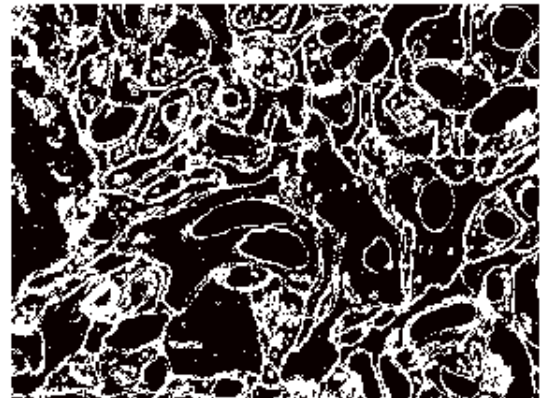
0.9.4 Check the performance of the thresholding

Let's create a confusion matrix to visualize the performance of the segmentation. A first step is to compute how many hits and misses our segmentation resulted in. In particular, looking at the four different cases that can occur in a binarization.

True Negative (118050)



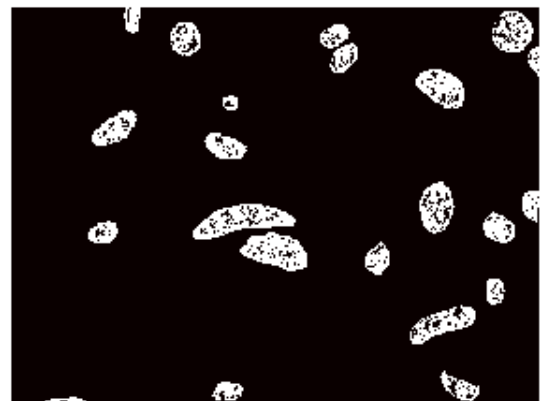
False Positive (61945)



False Negative (2932)



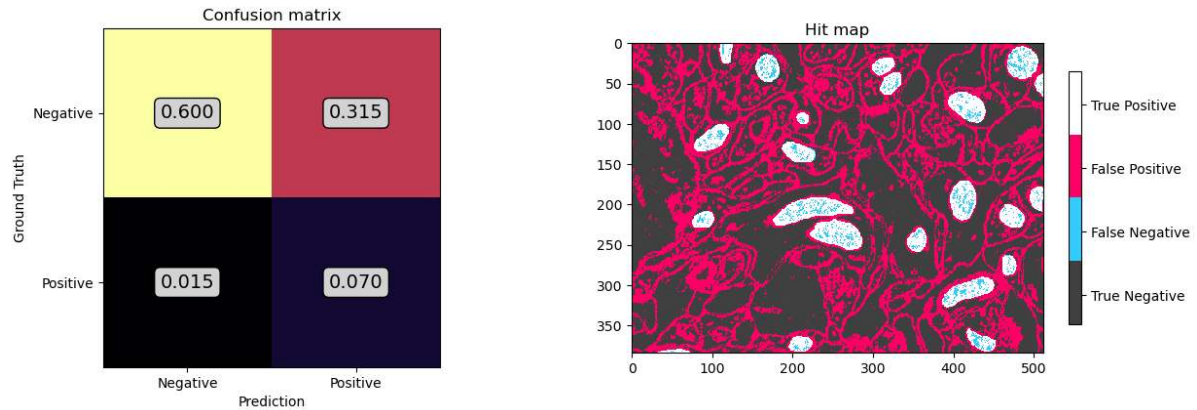
True Positive (13681)



0.9.5 The confusion matrix (revisited)

From the counts in the previous slide, we can now create a [Confusion matrix](#) and also look at the combined image of all the cases. In the hit map we can see white and gray as true segmentation and blue and magenta as false segmentations.

```
fig, ax=plt.subplots(1,2,figsize=(15,4.5))
ps.showHitMap(cell_seg,thresh_img,ax=ax) # this is a handy support function provided
↳with the material that comes with this notebook
```



There are many magenta colored pixels (false positive) in this example. The reason is that the mitochondria have the same intensity distribution as other structures in the image. Therefore, they will also be segmented as mitochondria in when you apply a single global threshold.

0.9.6 Apply Precision and Recall

We can use two further metrics to measure the performance of a segmentation method. These are based on the information in the confusion matrix like

Recall (sensitivity)

$$\frac{TP}{TP + FN}$$

Detected vs expected

Precision

$$\frac{TP}{TP + FP}$$

Expected vs detected

Recall is the sum the true positive relative to the number of positives in the mask or also as written here the sum of true positives and false negatives. Recall tells us how good the method is to find the correct label within the mask.

Precision is the sum of true positives relative to the total number of positives provided by our segmentation method. The precision tells us how much our method over segments the image.

Both recall and precision are scalar numbers in the interval $0 < m \leq 1$ where '1' is the ideal condition. More information about precision and recall can be found on [wikipedia](https://en.wikipedia.org/wiki/Precision_and_recall)

Let's compute precision and recall for our mitochondria example.

```
recall = out_results['True Positive'] / (out_results['True Positive']+out_results[
    ↪'False Negative'])
precision = out_results['True Positive'] / (out_results['True Positive']+out_results[
    ↪'False Positive'])
```

```
print('Recall: {0:0.2f}'.format(recall))
print('Precision: {0:0.2f}'.format(precision))
```

```
Recall: 0.82
Precision: 0.18
```

This result tells us that our segmentation was relatively good at finding the mitochondria, but also that this happened at the cost of many false positives.

0.10 Reciever Operating Characteristic (ROC)

The **Reciever Operating Characteristic (ROC)** curves are a very common tool for analyzing the performance of binary classification systems and there are a large number of tools which can automatically make them.

- The concept of the ROC curve was first developed for WW2 soldiers detecting objects in battlefields using radar.
- As we saw before, for a single threshold value 0.5, we were able to compute a single recall and precision.
- The ROC shows the relation between recall and precision for a segmentation model.

Let's compute the hit and miss statistics...

If we want to make an ROC curve we take a number of threshold values and compute the corresponding precision and recall values for each threshold. In the example below we scan threshold values from 0.1 to 0.9 and compute the hit and miss statistics to calculate the precision and recall.

	Threshold	Recall	Precision	False_Positive_Rate	True Negative \
0	0.100000	0.823512	0.180903	0.344148	118050
1	0.116327	0.823512	0.180903	0.344148	118050
2	0.132653	0.823512	0.180903	0.344148	118050

	False Positive	False Negative	True Positive	(True Negative, #009933)	\
0	61945	2932	13681		0
1	61945	2932	13681		0
2	61945	2932	13681		0

	(False Positive, #cc0000)	(False Negative, #cc0000)	\
0		179995	0
1		179995	0
2		179995	0

	(True Positive, #009933)
0	16613
1	16613
2	16613

... and plot the table.

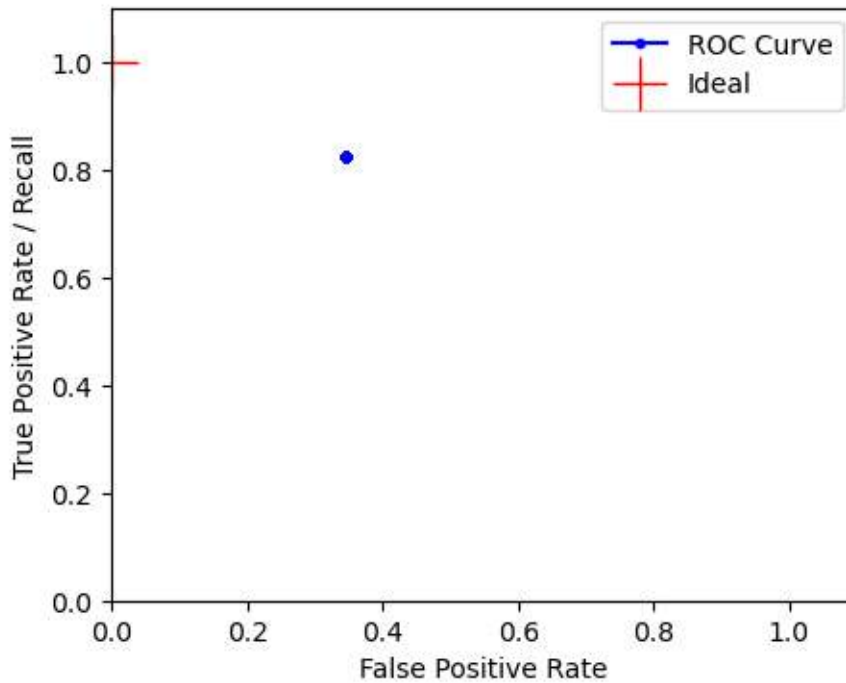
0.10.1 Making ROC Curves Easier

Here, we show how it is done with scikit-image.

Another way of showing the ROC curve (more common for data science applications rather than medical diagnosis) is using the True positive rate and False positive rate

- **True Positive Rate** (recall) = $TP / (TP + FN)$
- **False Positive Rate** = $FP / (FP + TN)$

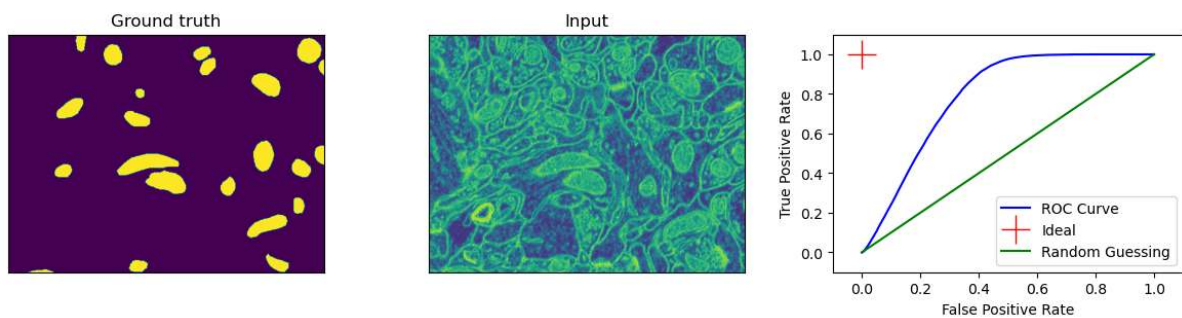
These show very similar information with the major difference being the goal is to be in the upper left-hand corner. Additionally random guesses can be shown as the slope 1 line. Therefore for a system to be useful it must lie above the random line.



ROC curve for mitochondria image segmentation

We only got a single point in the previous example. Now, let's see what happens when we scan the thresholds among the available gray levels in the images.

```
fpr, tpr, thresholds = roc_curve(y_true = cell_seg.ravel().astype(int),
                                y_score = cell_img.ravel())
```



The ROC curve for this case tells us that the true positives increase steadily but also that the cost is an increasing number of false positives. We have all true positives at about 0.5 false positives, beyond this point we only increase the number of misclassified pixels.

This curve indicates that a single threshold is maybe not the best approach for this image; we either get too many misclassified pixels or too few true positives.

0.10.2 Explore the impact of different filters on the ROC

We have already seen what the ROC curve looks like for the original data. Some weeks ago we learnt about a lot of filters and now it is time to see how these can be used in an attempt to improve the ROC. In this example we will compare the unfiltered image to:

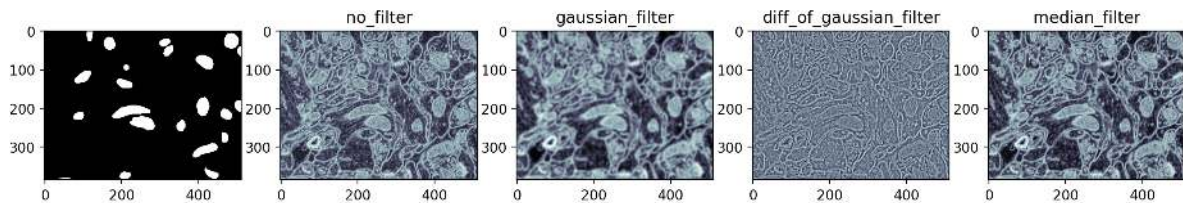
- Gaussian filter ($\sigma = 2$)
- Difference of Gaussian $x - G_{\sigma=3} * x$
- Median size 3x3

And see what performance improvements we can achieve

Let's produce some filtered images:

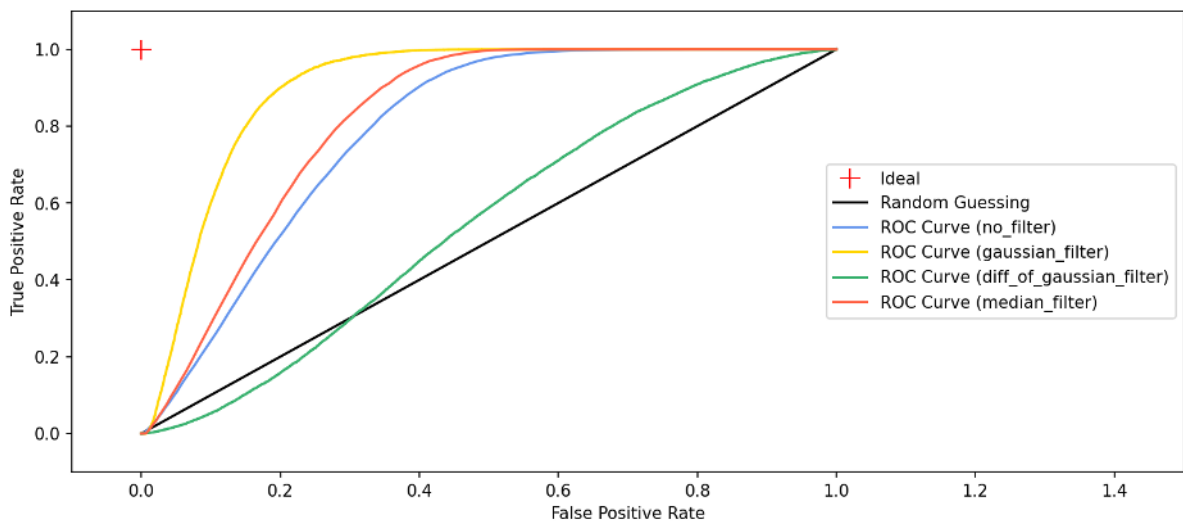
```
from skimage.filters import gaussian, median

def no_filter(x):          return x
def gaussian_filter(x):    return gaussian(x, sigma=2)
def diff_of_gaussian_filter(x): return x-gaussian(x, sigma=3)
def median_filter(x):      return median(x, np.ones((3, 3)))
```



We see that there are three low-pass filters and one high-pass filter.

ROC curves of filtered images



0.10.3 Area Under the Curve (AUC)

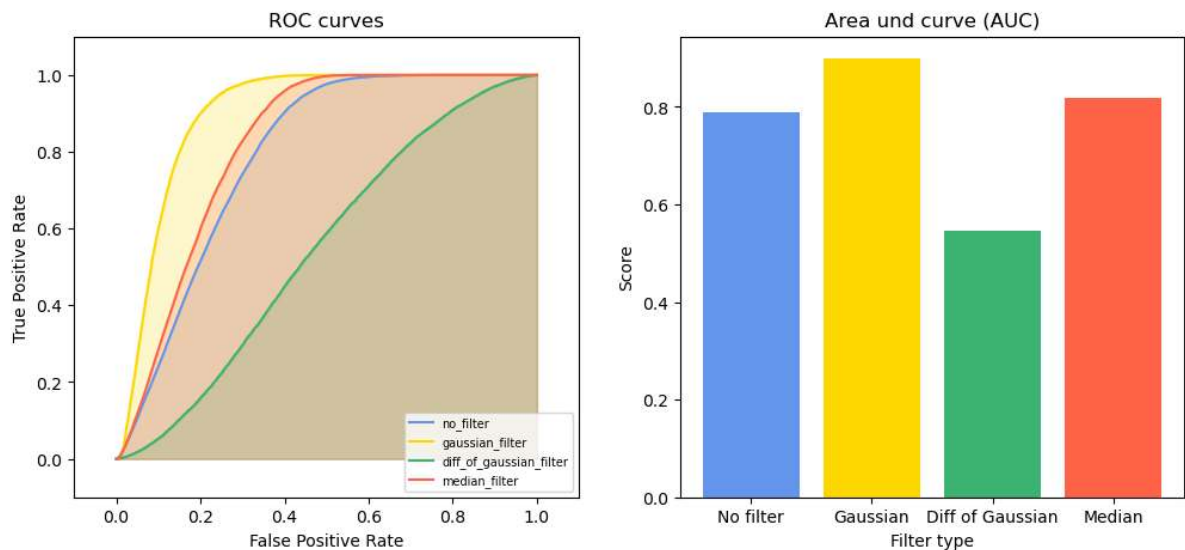
A curve nicely visualizes the behaviour of the segmentation method. It is, however, inconvenient when you want to compare the performance in an evaluation framework. Here, it is more convenient to use scalars.

We can then use the ROC curves to compare different filters (or even entire workflows), if the area is higher the approach is better.

Different approaches can be compared by *Area Under the Curve* (AUC) which is a scalar.

```
fig, ax = plt.subplots(1, 2, figsize=(12,5), dpi=100)
colors = ['cornflowerblue', 'gold', 'mediumseagreen', 'tomato']
scores = []
for color, c_filt in zip(colors, [no_filter, gaussian_filter, diff_of_gaussian_filter,
    median_filter]):
    fimg = c_filt(cell_img).ravel()
    fpr, tpr, thresholds = roc_curve(cell_seg.ravel().astype(int), fimg)
    scores.append(roc_auc_score(cell_seg.ravel().astype(int), fimg))
    ax[0].plot(fpr, tpr, '-', markersize=0.01, color=color, label='{}'.format(c_filt.__name__))
    ax[0].fill_between(fpr, tpr, 0, alpha=0.2, color=color)

ax[0].set_xlim(-0.1, 1.1); ax[0].set_ylim(-0.1, 1.1)
ax[0].set_xlabel('False Positive Rate'); ax[0].set_ylabel('True Positive Rate');
ax[0].set_title('ROC curves')
ax[0].legend(loc="lower right", fontsize=7);
names = ['No filter', 'Gaussian', 'Diff of Gaussian', 'Median']
ax[1].bar(names, scores, color=colors); plt.xlabel('Filter type'), ax[1].set_ylabel('Score')
ax[1].set_title('Area und curve (AUC)');
```



In this example, we see that segmentation after applying the Gaussian filter resulted in the greatest AUC. The reason is that the Gaussian filter blurs fine details better than the other alternatives. I.e., the thin lined network between the mitochondriae loses amplitude and also the speckled interior of the mitochondriae is smoother. The consequence of these effects is that the segmentation is less ambiguous and the AUC gets the highest score.

Armed with these tools we are ready to analyze the performance of the segmentation methods we develop to solve our image analysis tasks.

0.11 Segmenting multiple phases

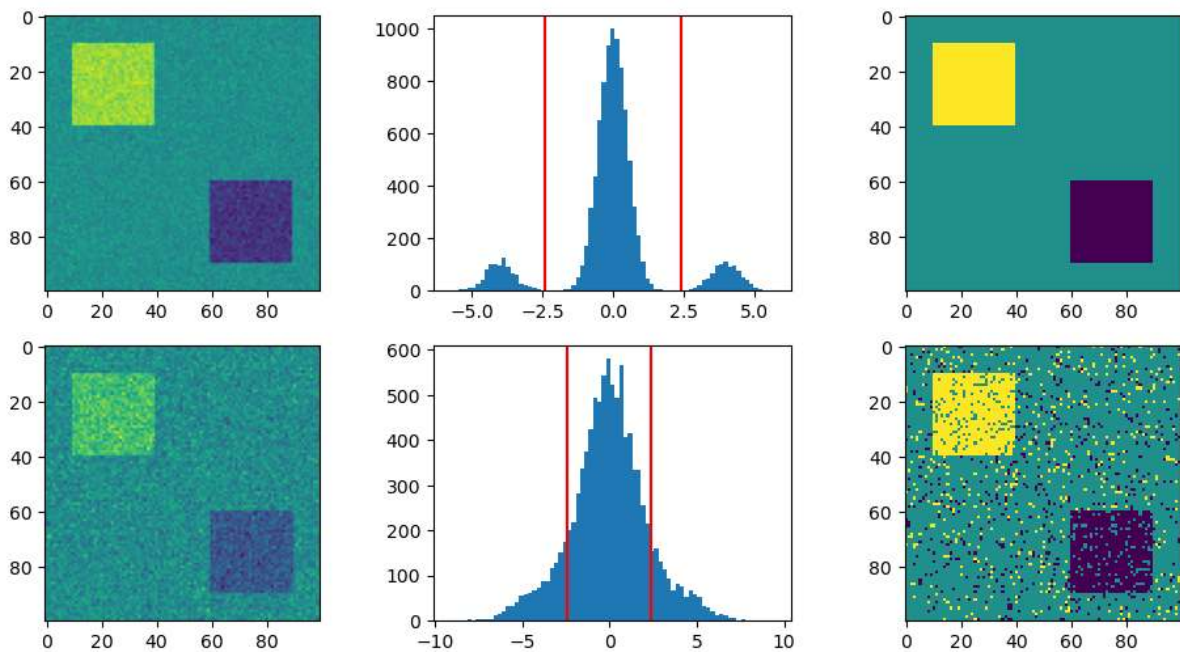
We have seen that we can separate an image into foreground and background using a single threshold. Let's see what happens when we want to separate more classes using more thresholds.

In theory, we could use $N - 1$ thresholds to separate N classes.

We have

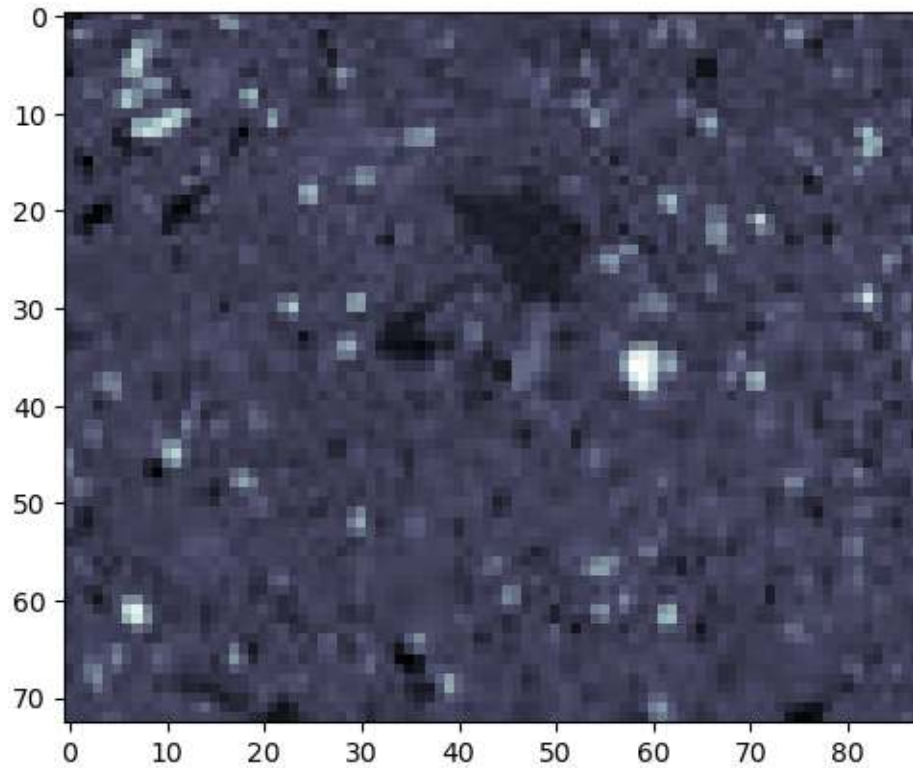
- three classes (A,B,C) and
- two thresholds (θ_0, θ_1)

$$I(x) = \begin{cases} A, & x \leq \theta_0 \\ B, & \theta_0 < x \leq \theta_1 \\ C, & \theta_1 < x \end{cases}$$



0.11.1 Multiple Phases example: Segmenting Shale

- Shale provided from Kanitpanyacharoen, W. (2012). *Synchrotron X-ray Applications Toward an Understanding of Elastic Anisotropy*.
- Here we have a shale sample measured with X-ray tomography with three different phases inside (clay, rock, and air).
- The model is that because the chemical composition and density of each phase is different they will absorb different amounts of x-rays and appear as different brightnesses in the image

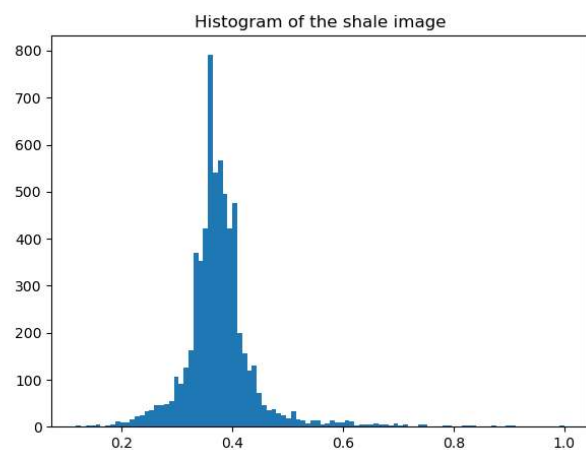
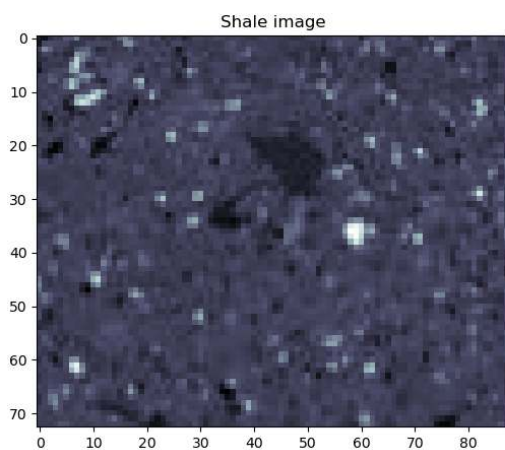


Finding three categories

Let's take a look at the histogram as always when we face a segmentation task...

Ideally we would derive 3 values for the thresholds based on a model for the composition of each phase and how much it absorbs, but that is not always possible or practical.

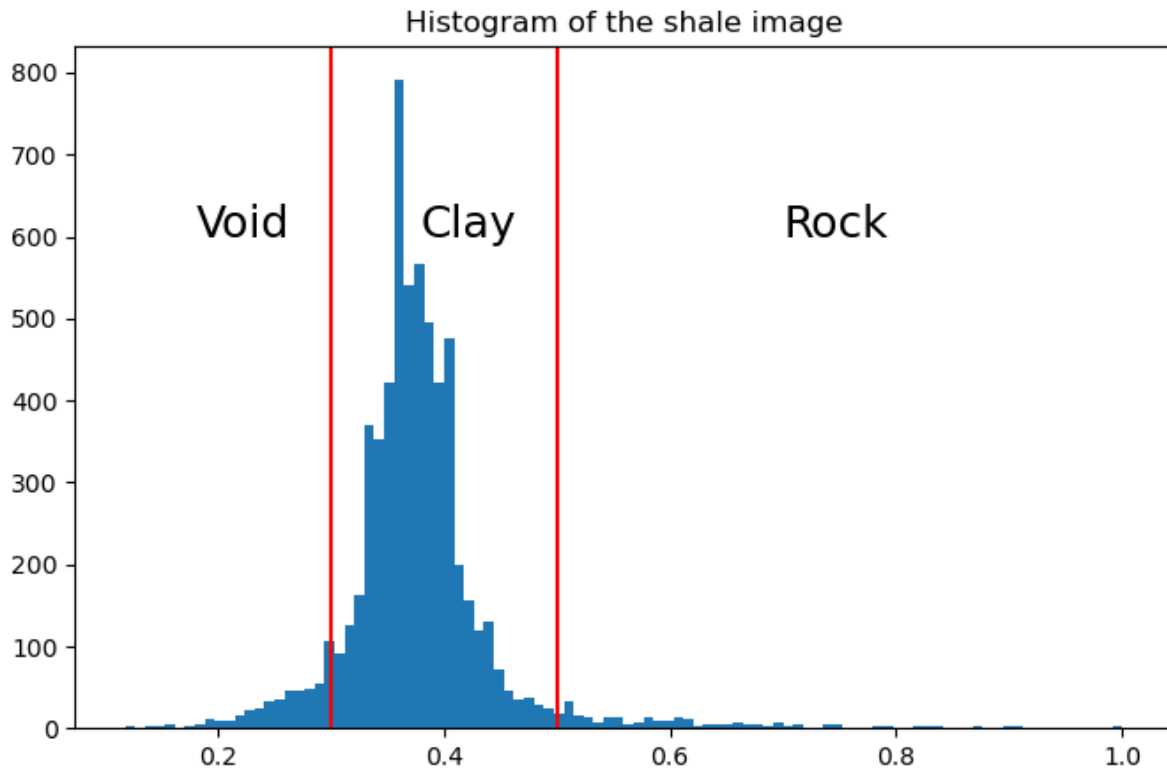
While there are 3 phases clearly visible in the image, the histogram is less telling (even after being re-scaled).



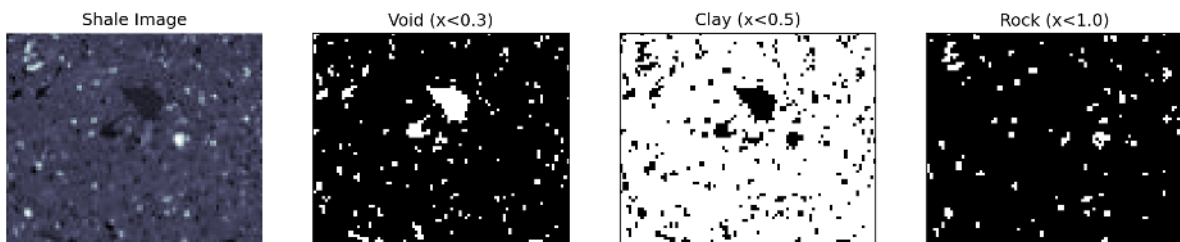
0.12 Multiple Segmentations

For this exercise we choose arbitrarily 3 ranges for the different phases and perform visual inspection

The relation can explicitly be written out as $I(x) = \begin{cases} \text{Void}, & 0 \leq x \leq 0.3 \\ \text{Clay}, & 0.3 < x \leq 0.5 \\ \text{Rock}, & 0.5 < x \end{cases}$



0.12.1 Segmentation result



Segmenting multiple phases is a non-trivial problem. In particular, when the edges in the image are smooth and at low SNR. We will look into these problems next week.

0.13 Implementation of thresholding

The implementations of basic thresholds and segmentations is very easy since it is a unary operation of a single image

$$f(I(\vec{x}))$$

How is this implemented with using a programming language?

In mathematical terms this is called a map and since it does not require information from neighboring voxels or images it can be calculated for each point independently (*parallel*). Filters on the other hand almost always depend on neighboring voxels and thus the calculations are not as easy to separate.

0.13.1 Implementation using script languages

Python (numpy) and Matlab

The simplest is a single threshold in numpy and Matlab:

```
thresh_img = gray_img > thresh
```

A more complicated threshold:

```
thresh_img = (gray_img > thresh_a) & (gray_img < thresh_b)
```

Python

The task is slightly more complicated when you use standard python. Here, you have to define a mapping function with a lambda to perform the thresholding.

```
thresh_img = map(lambda gray_val: gray_val>thresh, gray_img)
```

0.13.2 Implementation using traditional programming languages

In traditional programming languages you have to write some more code as there are no predefined functions that operate directly on arrays. This means, you'll have to implement the loops yourself.

Java

```
boolean[] thresh_img = new boolean[x_size*y_size*z_size];
for(int x=x_min ; x<x_max ; ++x)
    for(int y=y_min ; y<y_max ; ++y)
        for(int z=z_min ; z<z_max ; ++z)
        {
            int offset=(z*y_size+y)*x_size+x;
            thresh_img[offset]=gray_img[offset]>thresh;
        }
```

C++

```
bool* thresh_img = new bool[x_size*y_size*z_size];

for(int x=x_min ; x<x_max ; ++x)
  for(int y=y_min ; y<y_max ; ++y)
    for(int z=z_min ; z<z_max ; ++z)
    {
      int offset=(z*y_size+y)*x_size+x;
      thresh_img[offset]=gray_img[offset]>thresh;
    }
```

Alternative solution

- Image are stored as a sequence of numbers, not matter the number of dimensions.
- The pixel neighborhood is not considered.
- A single loop can be used!

```
int N=x_size*y_size*z_size;

bool* thresh_img = new bool[N];

for(int i=0 ; i<N ; ++i)
{
  thresh_img[i]=gray_img[i]>thresh;
}
```

The advantage is that we don't have to compute the location from the x,y,z coordinates which saves multiplications and additions for every pixel.

0.13.3 Summary segmentation

- Validation methods from data science help us to understand segmentation performance
- We need a ground truth for the validation
- Multiple thresholds are needed for multi-phase materials
- The implementation of the segmentation depends on the chosen language

0.14 Morphological image processing

The segmentation is rarely perfect!

By comparing a voxel with neighborhood voxels we can improve the results.

Like filtering the assumption behind morphological operations are

- nearby voxels in **real** images are related / strongly correlated with one another
- noise and imaging artifacts are less spatially correlated.

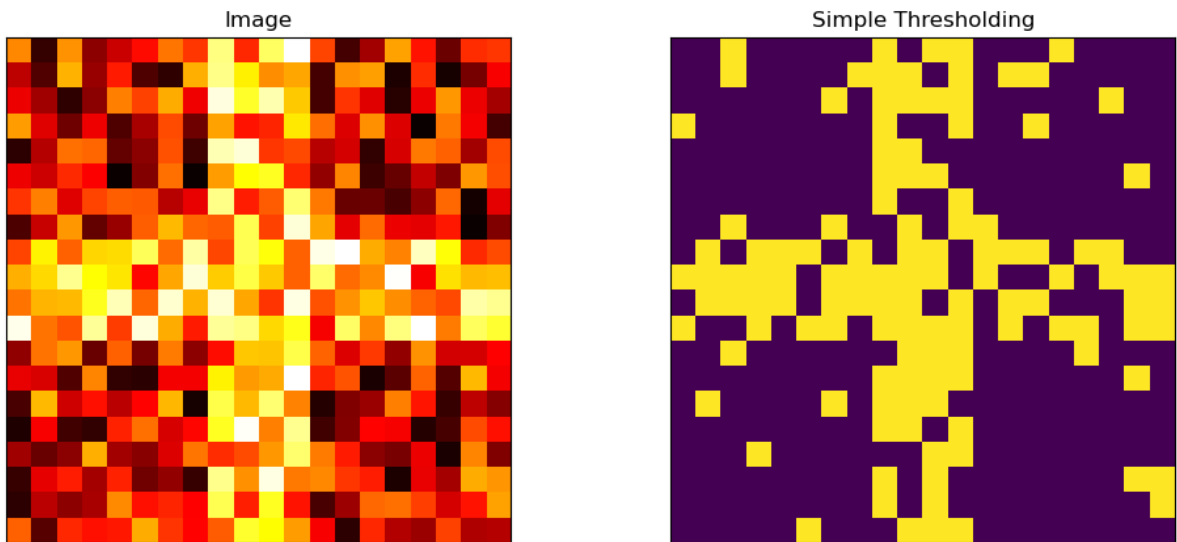
These steps are called morphological operations.

0.14.1 Segmenting noisy data

We return to the noisy image of a cross:

```
nx = 20
ny = 20
xx, yy = np.meshgrid(np.linspace(-10, 10, nx),
                     np.linspace(-10, 10, ny))
np.random.seed(2018)
cross_im = 1.1 * ((np.abs(xx) < 2) + (np.abs(yy) < 2)) + \
    np.random.uniform(-1.0, 1.0, size=xx.shape)

thimg = cross_im > 0.8 # Let's apply a threshold
```



We have a lot of misclassified pixels here!

We should probably apply a denoising filter here before trying to segment. This would improve the performance.

If this is not possible, we need to find alternative approaches. This is where morphological filters are relevant. They filter misclassifications using information from the pixel neighborhood. A misclassified pixel is mostly surrounded by correct pixels, they help to correct the current one.

0.14.2 Fundamentals: Neighborhood

A neighborhood consists of the pixels or voxels which are of sufficient proximity to a given point. There are a number of possible definitions which largely affect the result when it is invoked.

- A large neighborhood performs operations over larger areas / volumes
 - Computationally intensive
 - Can *smooth* relevant features
- A small neighborhood performs operations over small areas / volumes
 - Computationally cheaper
 - Struggles with large noise / filling large holes

Why do we need neighborhoods

The neighborhood is important for a large number of image and other (communication, mapping, networking) processing operations:

- filtering
- morphological operations
- component labeling
- distance maps
- image correlation based tracking methods

It is often called structuring element (`footprint` and `selem` are names in code), but has exactly the same meaning

Fundamentals: Neighbors in 2D

For standard image operations there are two definitions of neighborhood.

The direct adjacent pixels horizontally and vertically.

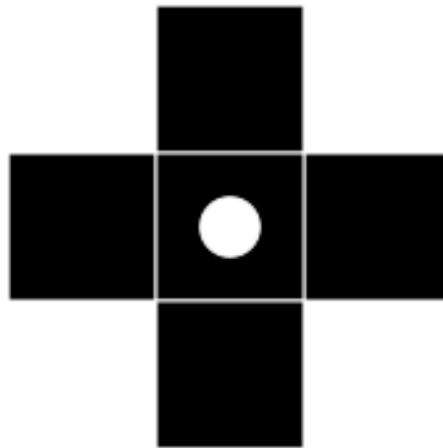


Fig. 1: 4-connected pixel neighborhood for 2D images.

or all pixels that surround the pixel.

The 4 and 8 adjacent neighbors shown below. Given the blue pixel in the center the red are the 4-adjacent and the red and green make up the 8 adjacent. We expand beyond this to disk, cross, vertical and horizontal lines

More 2D neighborhood shapes

The previous neighborhoods only considered the direct neighbors of the central pixel. We can however define larger neighborhood also with many different shapes.

Below you see a collection of different neighborhood shapes and sizes that can be used. Compact, convex neighborhood are mainly used for cleaning purposes, whereas lines and crosses can be used to enhance local structures.

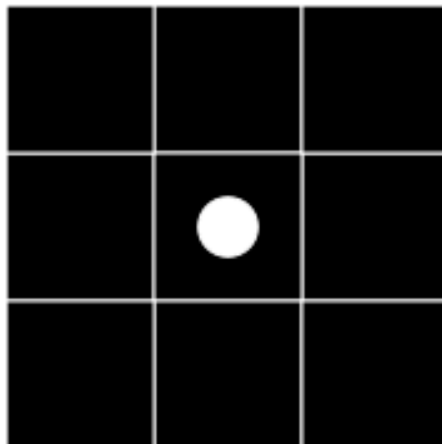
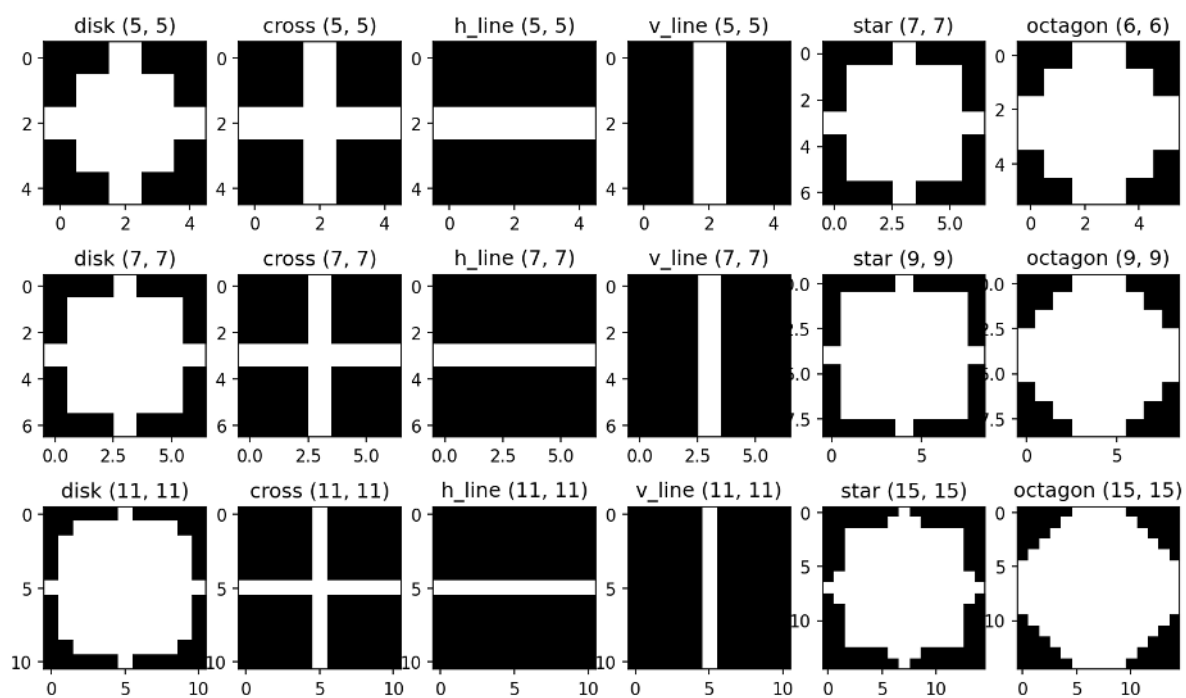


Fig. 2: 8-connected pixel neighborhood for 2D images.

Different neighborhood shapes and sizes



Fundamentals: Neighbors in 3D

Neighborhoods in 3D include the planes above and below the center pixel in addition to the neighbors in the same plane.

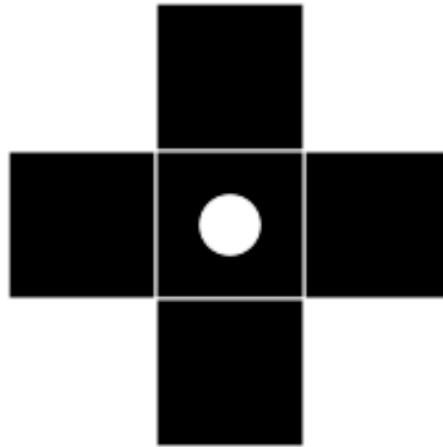


Fig. 3: 6-connected pixel neighborhood for 2D images.

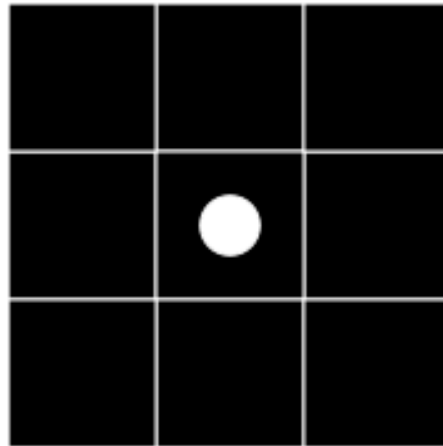


Fig. 4: 8-connected pixel neighborhood for 2D images.

0.14.3 Erosion and Dilation

Erosion

If any of the voxels in the neighborhood are 0/false then the central voxel will be set to 0

- Has the effect of peeling the surface layer off of an object

Dilation

If any of the voxels in the neighborhood are 1/true then the central voxel will be set to 1

- Has the effect of adding a layer onto an object
(dunking an strawberry in chocolate, adding a coat of paint to a car)

Applied Erosion and Dilation

Let's take a test image to demonstrate the effect of erosion and dilation. The image in the middle will be eroded and dilated. Using a 4-connected structure element.

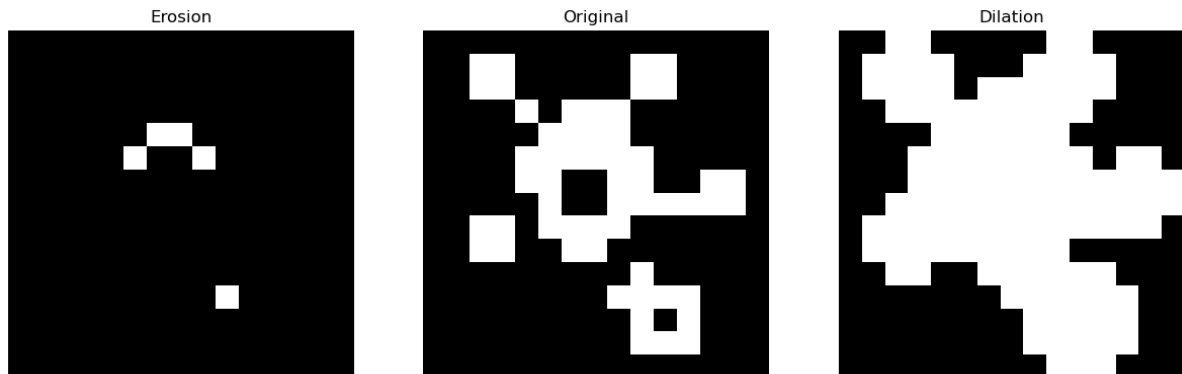
```
import skimage.morphology as morph
img=np.load('data/morphimage.npy')

selem = np.array([[0,1,0],
                  [1,1,1],
                  [0,1,0]])

dimg=morph.dilation(img,footprint=selem)
eimg=morph.erosion(img,footprint=selem)
```

```
fig, ax = plt.subplots(1,3,figsize=(15,6))

ax[0].imshow(eimg,cmap='gray'); ax[0].set_title('Erosion'), ax[0].axis('off');
ax[1].imshow(img,cmap='gray'); ax[1].set_title('Original'), ax[1].axis('off');
ax[2].imshow(dimg,cmap='gray'); ax[2].set_title('Dilation'), ax[2].axis('off');
```



We see that most of the test image is removed by the erosion and the gaps are filled with the dilation. Now, let's look at what is actually happening.

Dilation

We can use dilation to expand objects, for example a too-low threshold value leading to disconnected components

```
selem = np.array([[0,1,0],
                  [1,1,1],
                  [0,1,0]])

dimg=morph.dilation(img,footprint=selem)
```

```
fig, ax = plt.subplots(1,3,figsize=(15,6))

ax[0].imshow(img,cmap='gray');
ax[0].set(title='Original',xticks=[],yticks=[]);

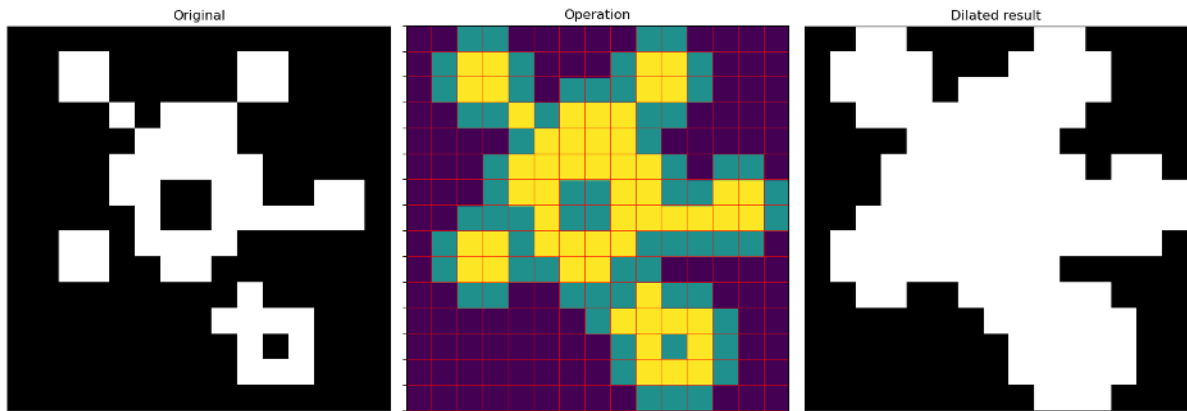
ax[1].imshow(img+dimg,cmap='viridis');
ax[1].set_xticks(np.arange(-0.5,img.shape[1],1)); ax[1].set_xticklabels([]);ax[1].set_
yticks(np.arange(-0.55,img.shape[0],1)); ax[1].set_yticklabels([])
```

(continues on next page)

(continued from previous page)

```
ax[1].grid(color='red', linestyle='-', linewidth=0.5); ax[1].grid(True); ax[1].set_
    title('Operation')

ax[2].imshow(dimg, cmap='gray');
ax[2].set(title='Dilated result', xticks=[], yticks=[]);
plt.tight_layout()
```

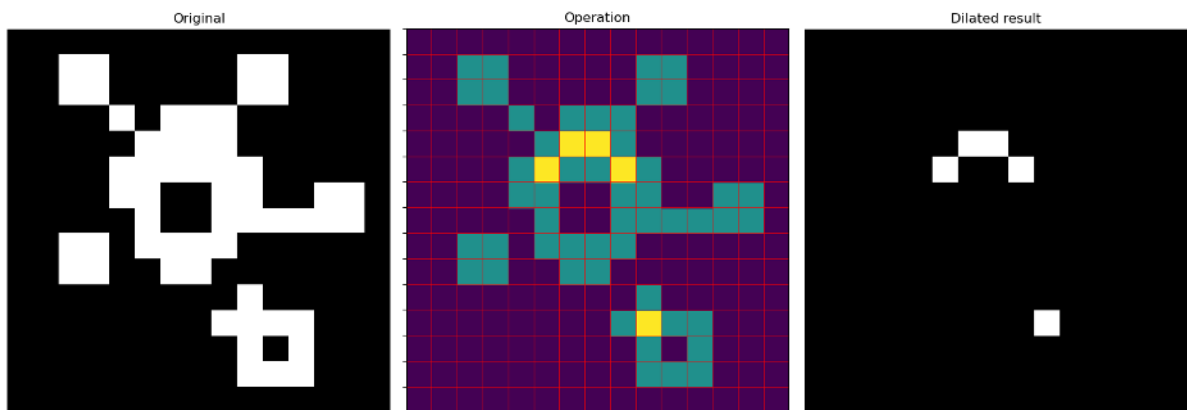


With the dilation, we add all the green pixels at the object boundaries. The dilation does not make any effect if the structure element is inscribed in the image item.

Erosion

Erosion performs the opposite task to the dilation by reducing the size of the objects in the image

```
selem = np.array([[0,1,0],
                  [1,1,1],
                  [0,1,0]])
eimg=morph.erosion(img, footprint=selem)
```



All boundary pixels are peeled off by the erosion, which only leaves few pixels remaining in the resulting image because there are only small structures here.

0.14.4 Opening and Closing

We saw before that erosion and dilations are too invasive. They either remove or add too much to the shapes which will bias the following quantitative analysis.

Erosion and dilation removes and adds a layer of pixels to the objects in the image.

This is not desired, combining them gives two new operations:

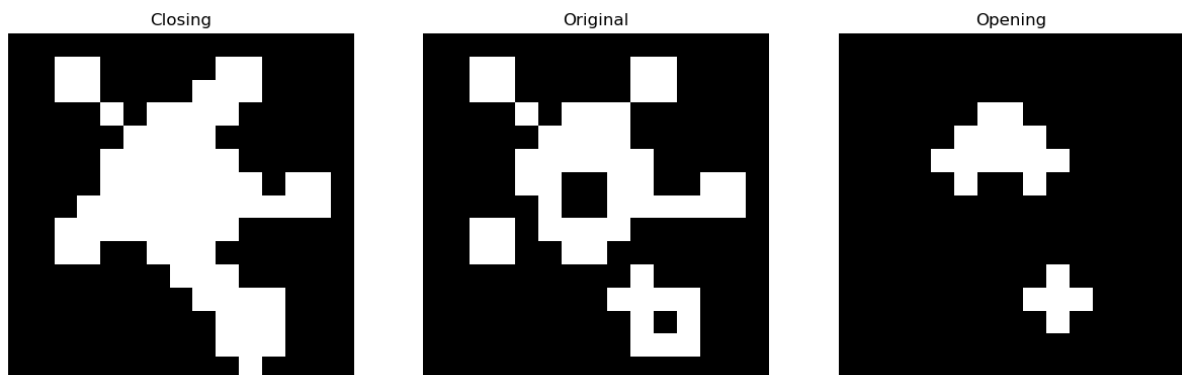
- Opening $\delta(\epsilon(f))$
- Closing $\epsilon(\delta(f))$

These operation rebuilds most of the objects after removing unwanted features.

By doing so you obtain results that are more reliable. The order in which you apply these operations matter. So, you'll have to experiment and observe which combination and also what sturcture elements are suited for your analysis.

```
selem = np.array([[0,1,0],[1,1,1],[0,1,0]])
oimg = morph.opening(img,footprint=selem)
cimg = morph.closing(img,footprint=selem)
```

```
fig, ax = plt.subplots(1,3,figsize=(15,6))
ax[0].imshow(cimg,cmap='gray'); ax[0].set_title('Closing'), ax[0].axis('off');
ax[1].imshow(img,cmap='gray'); ax[1].set_title('Original'), ax[1].axis('off');
ax[2].imshow(oimg,cmap='gray'); ax[2].set_title('Opening'), ax[2].axis('off');
```



We see that the opening and closing operations are less invasive than the erosion and dilation. The resulting structures resemble the original more than before.

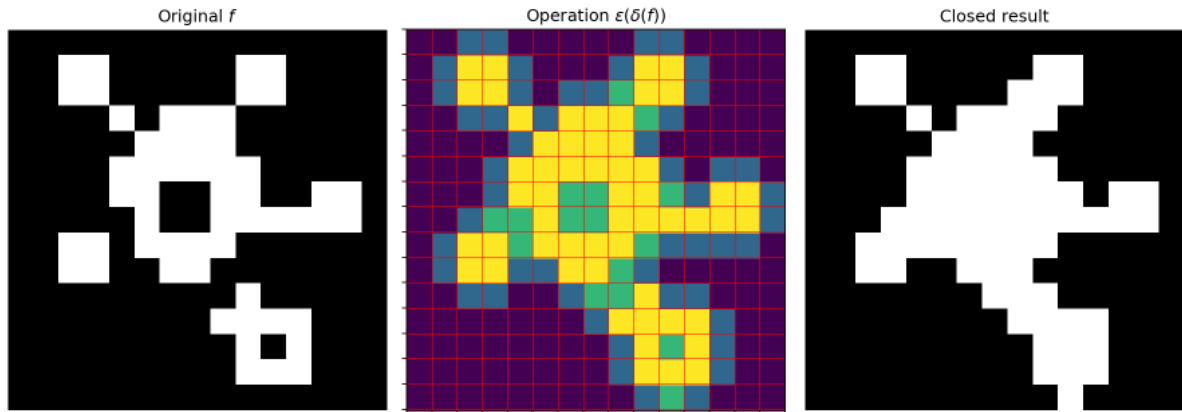
Morphological Closing

A dilation followed by an erosion operation

$$f \bullet b = \epsilon(\delta(f))$$

- Adds a layer and then peels a layer off
- Objects that are very close are connected when the layer is added and they stay connected when the layer is removed thus the image is `__close__`d
- A cube larger than one voxel will have the exact same volume after (conservative)

Closing is an operation you apply to remove false negatives in your image. The effect is that small holes in the objects are filled and gaps between large objects are connected.



Morphological opening

An erosion followed by a dilation operation $f \circ b = \delta(\epsilon(f))$

- Peels a layer off and adds a layer on
- Very small objects and connections are deleted in the erosion and do not return the image is thus **opened**
- A cube larger than several voxels will have the exact same volume after (conservative)

Opening is an operation you apply to remove false positives in your image. The effect is that small objects are erased and connections between large objects are removed.

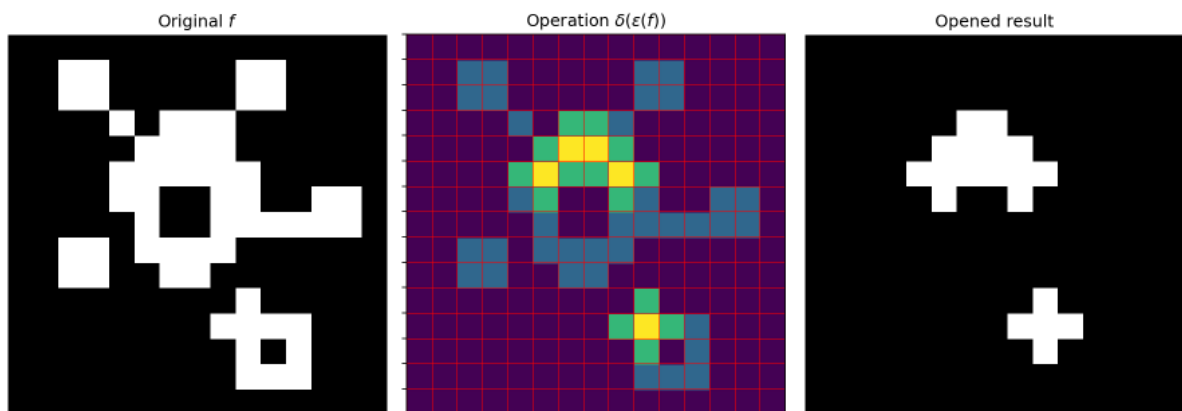
```
fig, ax = plt.subplots(1, 3, figsize=[12, 6], dpi=150)

ax[0].imshow(img, cmap='gray'); ax[0].axis('off');
ax[0].set_title('Original $f$')

ax[1].imshow(img+eimg+oimg, cmap='viridis'); ax[1].set_title('Operation $\delta(\epsilon(f))$')
ax[1].set_xticks(np.arange(-0.5, img.shape[1], 1)); ax[1].set_xticklabels([])
ax[1].set_yticks(np.arange(-0.55, img.shape[0], 1)); ax[1].set_yticklabels([])
ax[1].grid(color='red', linestyle='-', linewidth=0.5); ax[1].grid(True)

ax[2].imshow(oimg, cmap='gray'); ax[2].axis('off')
ax[2].set_title('Opened result')

plt.tight_layout()
```



Morphological filters on gray level images

The generalization of morphological filters on bi-level images is to apply them on gray level images.

For this we need the rank-n filter: Defined as sort the pixels covered by the structure element and select the n-th value.

Special cases of the rank filter

- Local max - dilation.
- Local min - erosion
- $N/2$ - Median

Mind exercise: See if this definition also works for bi-level images.

Further morphological operators - Top hats

Combining the opening ($f \circ b$) and closing ($f \bullet b$) operations with the original images:

The white top-hat

$$T_w = f - f \circ b$$

The black top-hat

$$T_b = f \bullet b - f$$

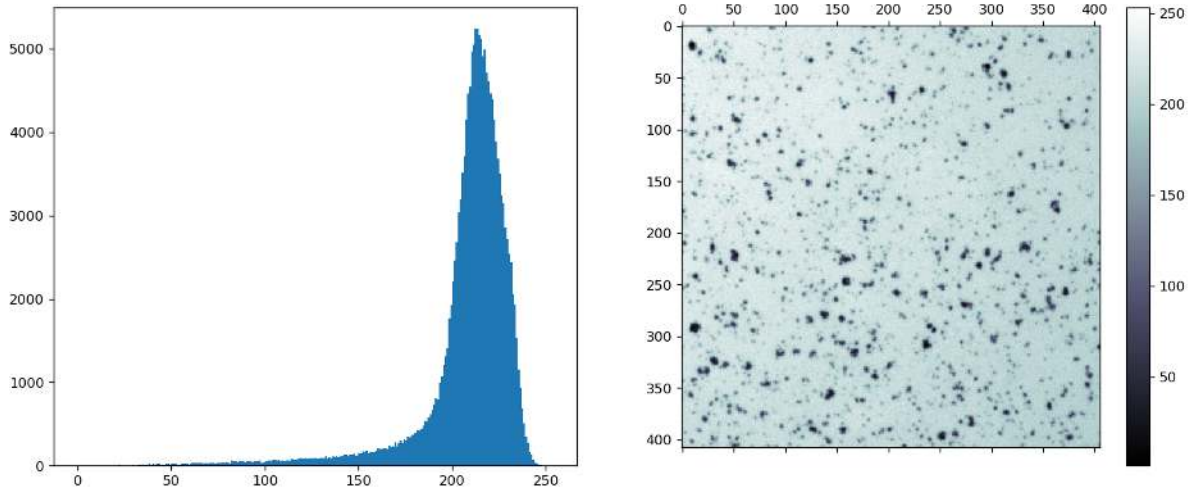
Using the tophat to flatten image

Let's return to the cell colony

The background in this image has an intensity gradient. This will have effect on the the segmentation performance if apply a single global threshold.

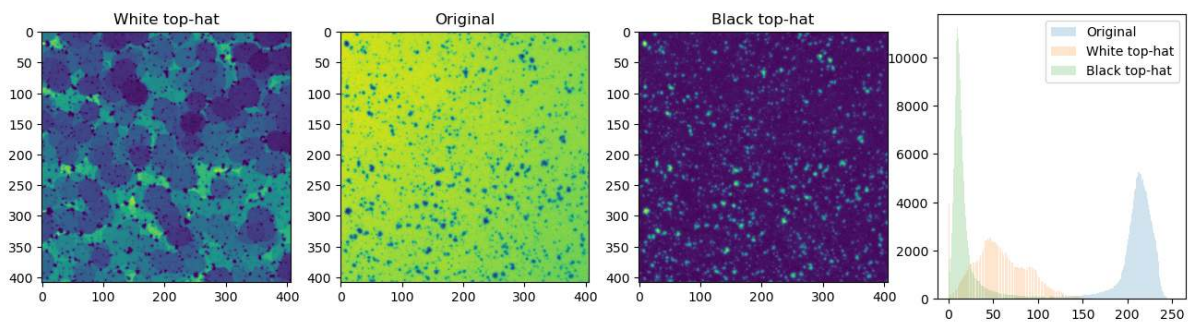
This problem should in principle be solved by normalization using a reference image with the illumination profile. This may for different reasons not always be available. In such cases, we have to create a background that can be used to level out the background gradient.

The top-hat is one such operation.



Applying the top-hat on the cell data

```
se = morph.disk(15)
tw = cell_img - morph.dilation(morph.erosion(cell_img, footprint=se), footprint=se)
tb = morph.erosion(morph.dilation(cell_img, footprint=se), footprint=se) - cell_img
```



The black and white top-hats behave quite differently

- The black removes dark regions with closing operation
- The white removes bright regions with the opening operator

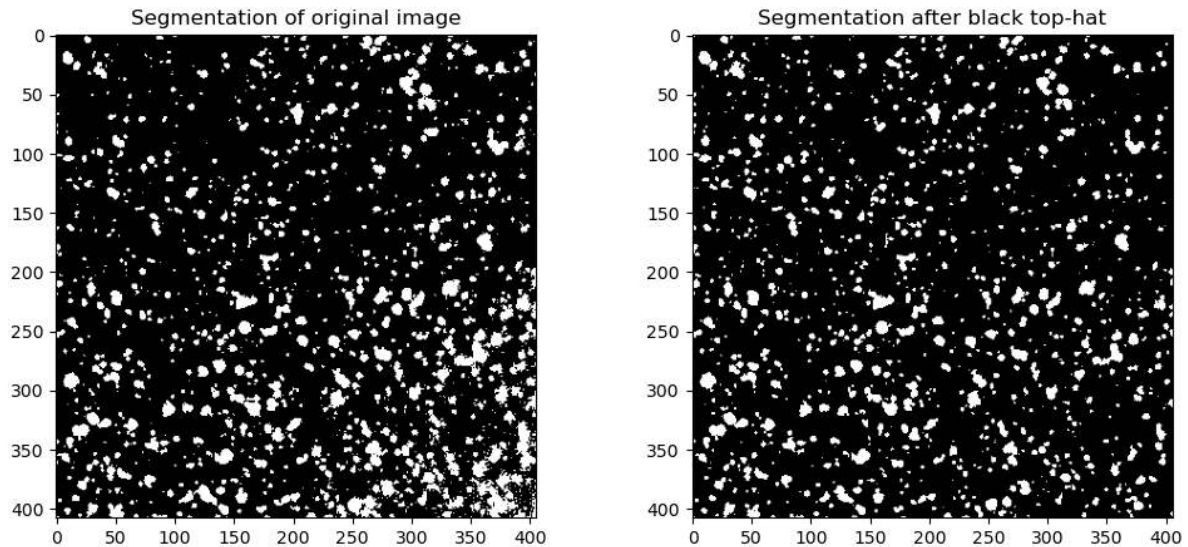
Both do strive to remove a background bias.

It is beneficial to use the black top-hat in this example as the dark regions are smaller than the bright. Therefore, we can use a smaller structure element and process the image faster. The white top-hat would here require a huge structure element.

You can also see that the gradient is essentially eliminated in the black top-hat image.

Comparing thresholding after Black tophat

We just saw that the gradient was nearly corrected with the black top-hat. Here is the resulting segmentation compared with the original we saw earlier in this lecture.



We already observed that the cell size changed in the top-left to bottom-right diagonal. Some cells were barely detected. After black top hat correction we see a more homogeneous size distribution across the image.

Which type of top-hat you choose depends on the image composition. In this case it was most efficient to use the black top-hat. It required a smaller structure element than the white top-hat. There, we would have to increase the structure element disc radius by more than the double to achieve the same effect.

0.15 Pitfalls with Segmentation

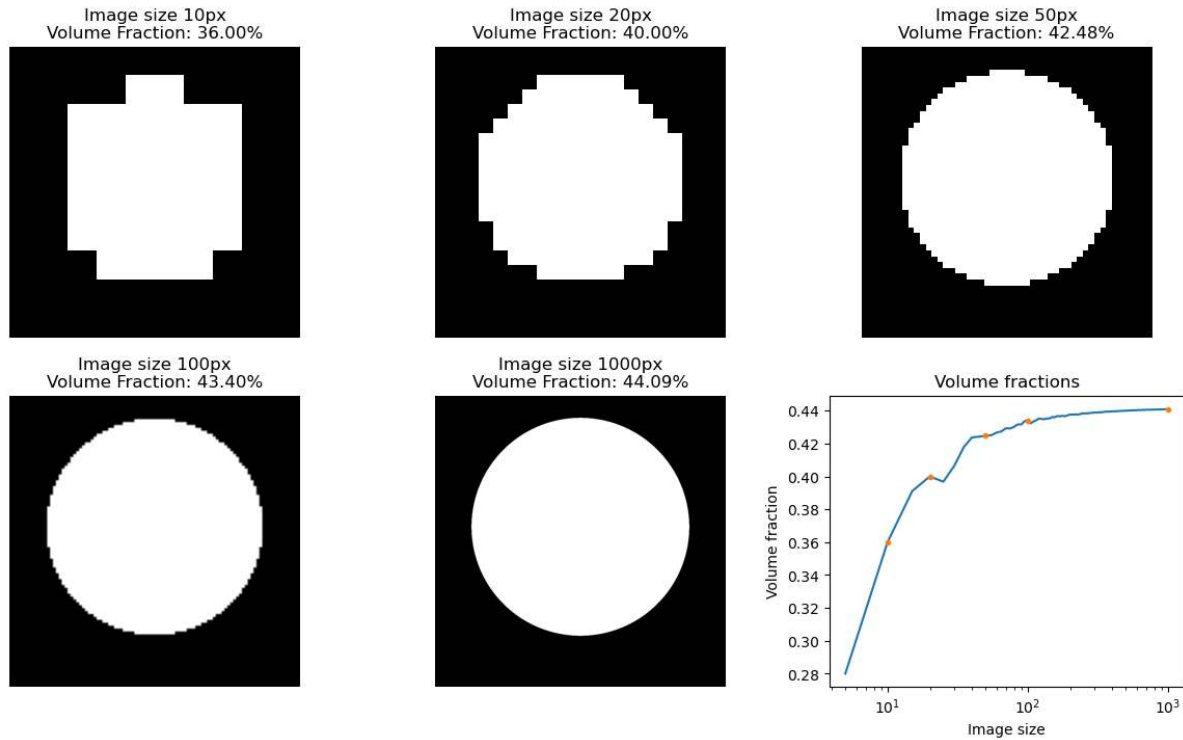
0.15.1 Partial Volume Effect

- The **partial volume effect** is the name for the effect of discretization on the image into pixels or voxels.
- Surfaces are complicated, voxels are simple boxes which make poor representations
- Many voxels are only partially filled, but only the voxels on the surface
- Removing the first layer alleviates issue

0.15.2 Thresholding structures

What happens when we threshold objects of different sizes?

In this example we create a series of discs on different image sizes from 5 up to 1000 pixels. The discs have the same relative size and are supposed to fill that same area fraction.



Here you can see that the small objects are very pixelated and almost doesn't resemble a disk. When object size increases we see that the object looks more and more like a round disk. We also see that the volume fraction increases towards the value the resembles the volume of a true disc.

0.15.3 When is a sphere really a sphere?

We just saw that a 2D disc can be very pixelated for small radii. The same applies in 3D. In this example, you can see what a sphere looks like. The first two examples doesn't really look like a sphere, while the last one starts to look like a sphere. The plot in the last panel shows the volume error for different discrete spheres. At a radius of about 10 pixels the error is below one percent.

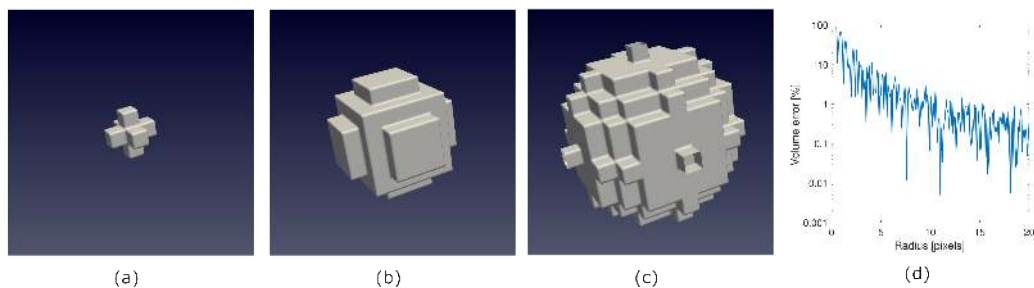


Fig. 1: Discrete spheres with increasing radius.

Using the volume ratio between analytical and voxelized sphere

$$V_{error} = \frac{V_{discrete}}{V_{Analytical}}$$

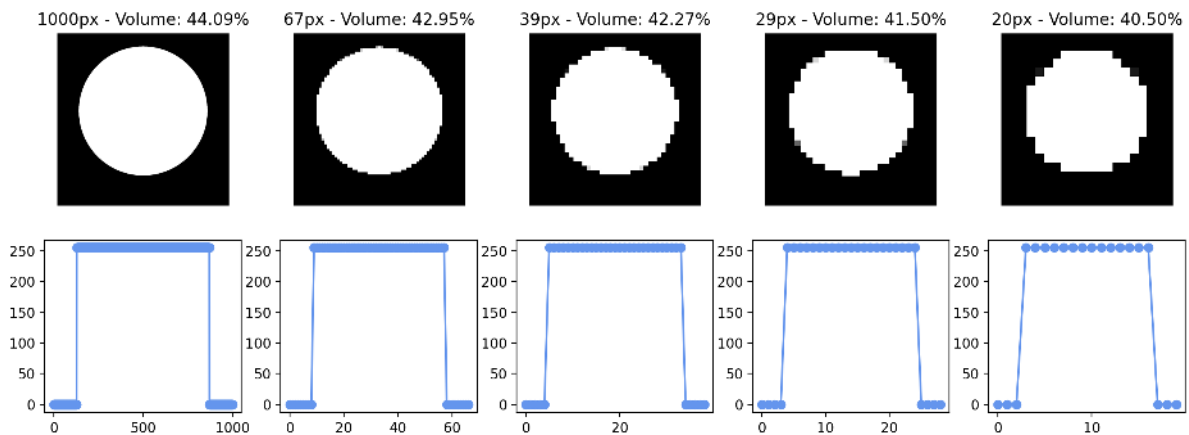
Kaestner et al. 2017

What we are learning from this study is that there is a difference between detecting a basic feature and really representing its true shape. Detection should in principle be possible within a few pixels if the SNR is sufficiently high.

0.15.4 Rescaling

Sometimes, we want to downscale the image when it is too large. This is mostly done due to problems of fitting the images into memory or to speed up the processing. Rescaling should generally be done on gray scale images to avoid visible partial volume effects, which means that pixels don't have only two values anymore at the edges.

In this example we rescale images from 500x500 down to 15x15 that the apparent volume fraction changes at the edges in some positions.



The intermediate values are in particular visible in the profiles from downsizing from 500 pixel to 20 and 34 pixels.

What happens at downsampling

The object rarely fills entire pixels at the edges.

This is relevant already when you acquire the images but also when you downsample.

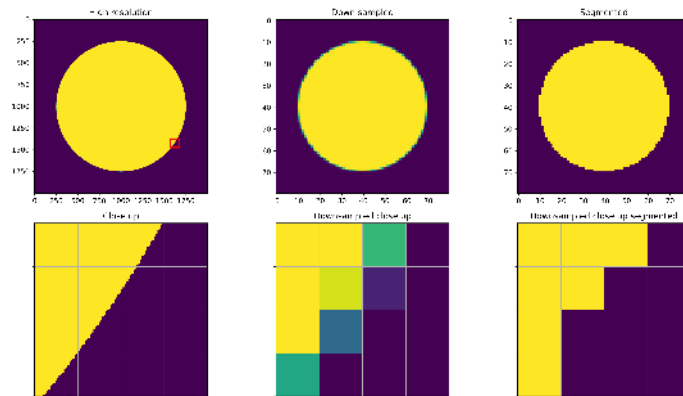


Fig. 2: Discrete spheres with increasing radius.

The figure shows that some of the edge pixels in the downsampled image are not completely inside or outside the edge. These pixels will obtain an intensity between foreground and background, which you see results in a blurred edge. The will then be less smooth in the thresholded image.

0.16 Summary

In today's lecture we have looked into

- The image formation process and how it relates to the segmentation problem.
- How the histogram can be used to decide how to segment an image.
- Evaluation of segmentation performance.
- The basic operations of morphological image processing
 - Using morphological operations to clean up segmented images
- Pitfall with segmentation - partial volume effects.

0.16.1 Next week

Automated segmentation methods